



Vitis High-Level Synthesis User Guide (UG1399)

Optimizing Techniques and Troubleshooting Tips

Understanding High-Level Synthesis Scheduling and Binding

Optimizing Logic

Optimizing AXI System Performance

Managing Area and Hardware Resources

Unrolling Loops in C++ Classes

Limitations of Control-Driven Task-Level Parallelism

Limitations of Pipelining with Static Variables

Optimizing Techniques and Troubleshooting Tips

This section outlines the various optimization techniques you can use to direct AMD Vitis™ HLS to produce a micro-architecture that satisfies the desired performance and area goals. Using Vitis HLS, you can apply different optimization directives to the design, including:

- Pipelining tasks, allowing the next execution of the task to begin before the current execution is complete.
- Specifying a target latency for the completion of functions, loops, and regions.
- Specifying a limit on the number of resources used.
- Overriding the inherent or implied dependencies in the code to permit specific operations. For example, if it is acceptable to discard or ignore the initial data values, such as in a video stream, allow a memory read before write if it results in better performance.
- Specifying the I/O protocol to ensure function arguments can be connected to other hardware blocks with the same I/O protocol.

 **Note:** Vitis HLS automatically determines the I/O protocol used by any sub-functions. You *cannot* control these ports except to specify whether the port is registered.

It helps to understand the process used to synthesize RTL hardware description from C/C++ source code. The [Understanding High-Level Synthesis Scheduling and Binding](#) describes some of the important details of this process to help you better understand how you can optimize for it.

You can add optimization directives directly into the source code as compiler pragmas using various HLS pragmas, or you can use directives as configuration commands as discussed in [Adding Pragmas and Directives](#). The following table lists the optimization directives provided by the HLS compiler as either pragma or configuration commands.

Table: Vitis HLS Optimization Directives

Directive	Description
-----------	-------------

Directive	Description
AGGREGATE	The AGGREGATE pragma is used for grouping all the elements of a struct into a single wide vector to allow all members of the struct to be read and written to simultaneously.
ALIAS	The ALIAS pragma enables data dependence analysis in Vitis HLS by defining the distance between multiple pointers accessing the same DRAM buffer.
ALLOCATION	Specify a limit for the number of operations, implementations, or functions used. This can force the sharing of hardware resources and may increase latency.
ARRAY PARTITION	Partitions large arrays into multiple smaller arrays or into individual registers, to improve access to data and remove block RAM bottlenecks.
ARRAY_RESHAPE	Reshape an array from one with many elements to one with greater word-width. Useful for improving block RAM accesses without using more block RAM.
BIND_OP	Define a specific RTL implementation for an operation.
BIND_STORAGE	Define a specific implementation for a storage element, or memory, in the RTL.
DATAFLOW	Enables task level pipelining, allowing functions and loops to execute concurrently. Used to optimize throughput and/or latency.
DEPENDENCE	Used to provide additional information that can overcome loop-carried dependencies and allow loops to be pipelined (or pipelined with lower intervals).
DISAGGREGATE	Break a struct down into its individual elements.
EXPRESSION_BALANCING	Disables automatic expression balancing to be turned off.
INLINE	Inlines a function, removing function hierarchy at this level. Used to enable logic optimization across function

Directive	Description
	boundaries and improve latency/interval by reducing function call overhead.
INTERFACE	Specifies how RTL ports are created from the function description.
LATENCY	Allows a minimum and maximum latency constraint to be specified.
LOOP_FLATTEN	Allows nested loops to be collapsed into a single loop with improved latency.
LOOP_MERGE	Merge consecutive loops to reduce overall latency, increase sharing and improve logic optimization.
LOOP_TRIPCOUNT	Used for loops which have variables bounds. Provides an estimate for the loop iteration count. This has no impact on synthesis, only on reporting.
OCCURRENCE	Used when pipelining functions or loops, to specify that the code in a location is executed at a lesser rate than the code in the enclosing function or loop.
PERFORMANCE	Specify the desired transaction interval for a loop and let the tool to determine the best way to achieve the result.
PIPELINE	Reduces the initiation interval by allowing the overlapped execution of operations within a loop or function.
PROTOCOL	This commands specifies a region of code, a protocol region, in which no clock operations will be inserted by Vitis HLS unless explicitly specified in the code.
RESET	This directive is used to add or remove reset on a specific state variable (global or static).
STABLE	Indicates that a variable input or output of a dataflow region can be ignored when generating the synchronizations at entry and exit of the dataflow region.

Directive	Description
STREAM	Specifies that a specific array is to be implemented as a FIFO or RAM memory channel during dataflow optimization. When using <code>hls::stream</code> , the STREAM optimization directive is used to override the configuration of the <code>hls::stream</code> .
TOP	The top-level function for synthesis is specified in the project settings. This directive may be used to specify any function as the top-level for synthesis. This then allows different solutions within the same project to be specified as the top-level function for synthesis without needing to create a new project.
UNROLL	Unroll for-loops to create multiple instances of the loop body and its instructions that can then be scheduled independently.

In addition to the optimization directives, Vitis HLS provides a number of configuration commands that can influence the performance of synthesis results. Details on using configurations commands can be found in HLS Config File Commands. The following table reflects some of these commands.

Table: Vitis HLS Configurations

GUI Directive	Description
Array Partition Configuration	Determines how arrays are partitioned, including global arrays and if the partitioning impacts array ports.
Compile Options	Controls synthesis specific optimizations such as the automatic loop pipelining and floating point math optimizations.
Dataflow Configuration	Specifies the default memory channel and FIFO depth in dataflow optimization.
Interface Configuration	Controls I/O ports not associated with the top-level function arguments and allows unused ports to be eliminated from the final RTL.

GUI Directive	Description
Operator Configuration	Configures the default latency and implementation of specified operations.
RTL Configuration	Provides control over the output RTL including file and module naming, and reset controls.
Schedule Setting	Determines the effort level to use during the synthesis scheduling phase and the verbosity of the output messages
Storage Configuration	Configures the default latency and implementation of specified storage types.
Unroll Setting	Configures the default tripcount threshold for unrolling loops.

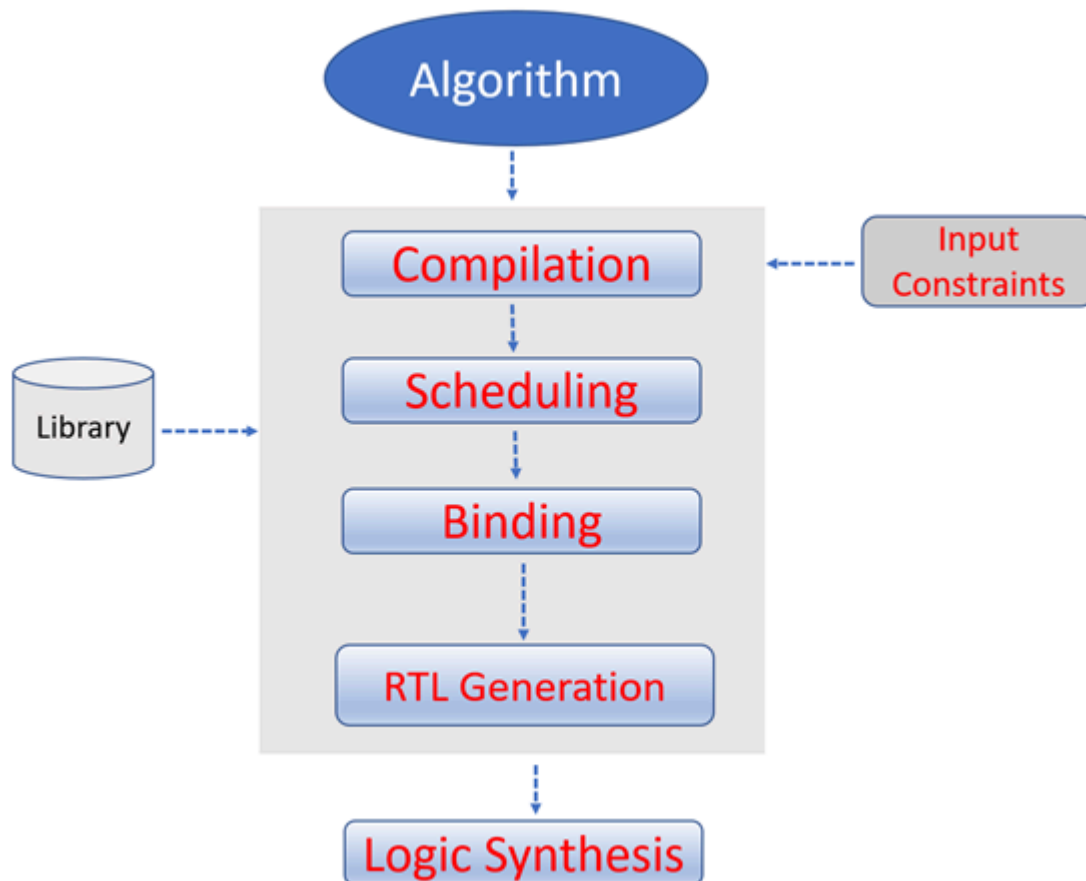
Understanding High-Level Synthesis Scheduling and Binding

High-Level Synthesis tools transform an untimed high-level specification into a fully timed implementation. During this transformation, a custom architecture is implemented to meet the specification requirements. The architecture generated contains the data path, control logic, memory interfaces, and how the RTL communicates with the external world. A data path consists of a set of storage elements such as (registers, register files, or memories), a set of functional units (such as ALUs, multipliers, shifters, and other custom functions), and interconnect elements (such as tristate drivers, multiplexers, and buses). Each component can take one or more clock cycles to execute, can be pipelined, and can have input or output registers. In addition, the entire data path and controller can be pipelined in several stages.

The designers should invest the early part of the project in redefining the architecture of the algorithm to meet the performance while keeping the algorithm at a higher level. For any specific HLS tool, there are design principles and best practices that are required to be followed to generate the optimized RTL that meets the expected performance.

The HLS Tool executes the following tasks as shown in the diagram below.

Figure: HLS Steps



1. Compile the algorithm written to meet specifications: This step includes several code optimizations such as dead-code elimination, constant folding, reporting unsupported constructs, etc.
2. Schedule the operations for given clock cycles:
 - a. The "Schedule" phase determines which operations occur during each clock cycle based on:
 - When an operation's dependencies have been satisfied or are available.
 - The length of the clock cycle or clock frequency.
 - The time it takes for the operation to complete, as defined by the target device. More operations can be completed in a single clock cycle for longer clock periods. Some operations might need to be implemented as multi-cycle resources. HLS automatically schedules operations over more clock cycles
 - The available resources.
 - Incorporation of any user-specified optimization directives.
 - b. During the "Schedule" phase, the tool determines what operator will execute in a given cycle and how many of these components are needed. The next step determines what operation binds to what resource.

3. Bind the operations to the functional components and variables to the storage elements
 - a. The binding task assigns hardware resources to implement each scheduled operation and maps operators (such as addition, multiplication, and shift) to specific RTL implementations. For example, a mult operation can be implemented in RTL as a combinational or pipelined multiplier.
 - b. The binding task assigns memories, registers, or combinations of these to the array variables inside the function to meet the desired performance.
 - c. If multiple operations use the same resource, this step can perform the resource sharing if not used in the same cycle.
4. Control logic extraction creates a finite state machine (FSM) that sequences the operations in the RTL design according to the defined schedule.
5. Creates the logic to communicate with the external world: The RTL generated will be communicating with the external world like streaming data from the external port or start/stop logic or accessing external memory.
6. Finally, generate the RTL architecture

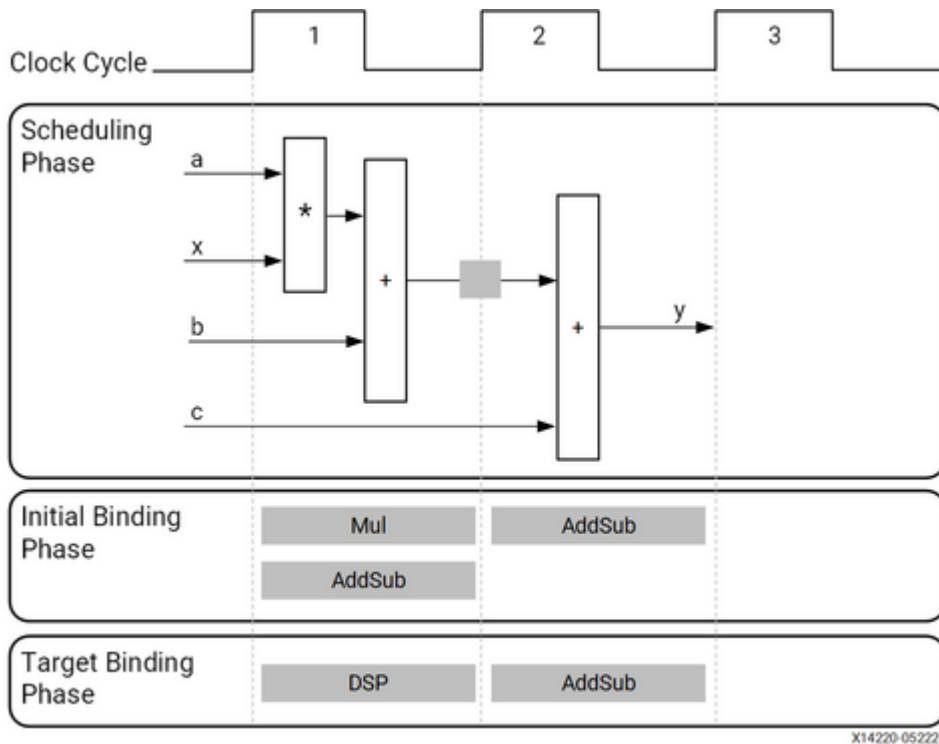
The next section walks through conceptually, how an HLS tool in general schedules the operators based on input constraints like a clock cycle and binds them to available hardware resources.

Scheduling and Binding Example

The following figure shows an example of the scheduling and binding phases for this code example:

```
int foo(char x, char a, char b, char c) {
    char y;
    y = x*a+b+c;
    return y;
}
```

Figure: Scheduling and Binding Example



In the scheduling phase of this example, high-level synthesis schedules the following operations to occur during each clock cycle:

- First clock cycle: Multiplication and the first addition
- Second clock cycle: Second addition, if the result of the first addition is available in the second clock cycle, and output generation

Note: In the preceding figure, the square between the first and second clock cycles indicates when an internal register stores a variable. In this example, high-level synthesis only requires that the output of the addition is registered across a clock cycle. The first cycle reads *x*, *a*, and *b* data ports. The second cycle reads data port *c* and generates output *y*.

In the final hardware implementation, high-level synthesis implements the arguments to the top-level function as input and output (I/O) ports. In this example, the arguments are simple data ports. Because each input variable is a char type, the input data ports are all 8-bits wide. The function return is a 32-bit int data type, and the output data port is 32-bits wide.

Important: The advantage of implementing the C code in the hardware is that all operations finish in a shorter number of clock cycles. In this example, the operations complete in only two clock cycles. In a central processing unit (CPU), even this simple code example takes more clock cycles to complete.

In the initial binding phase of this example, high-level synthesis implements the multiplier operation using a combinational multiplier (Mul) and implements both add operations using a combinational adder/subtractor (AddSub).

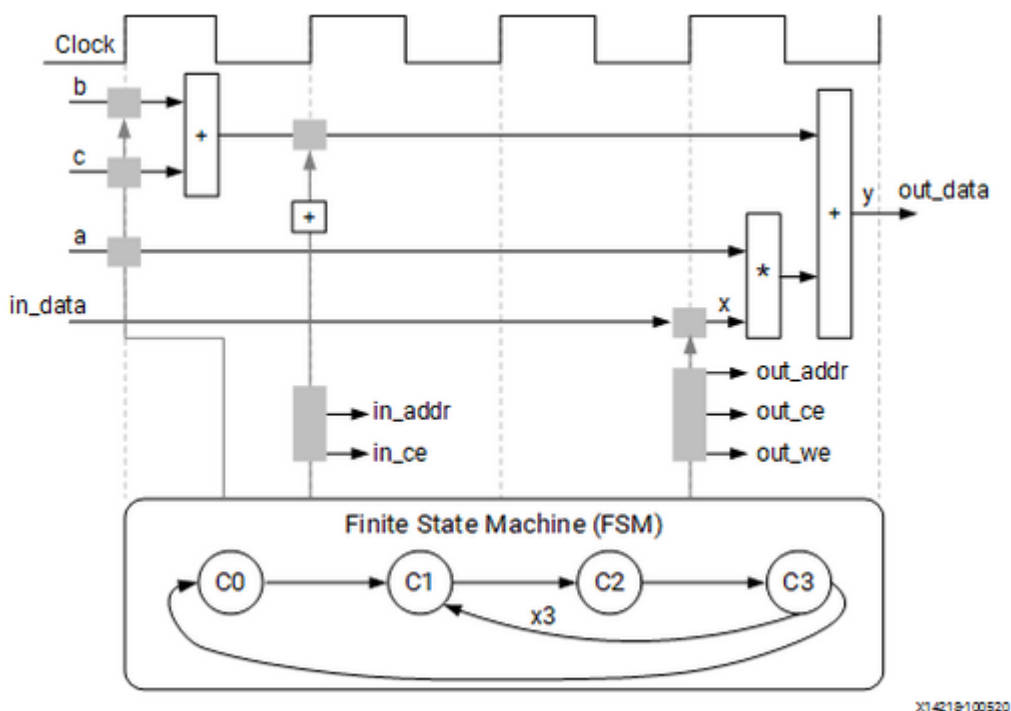
In the target binding phase, high-level synthesis implements both the multiplier and one of the addition operations using a DSP module resource. Some applications use many binary multipliers and accumulators that are best implemented in dedicated DSP resources. The DSP module is a computational block available in the FPGA architecture that provides the ideal balance of high-performance and efficient implementation.

Extracting Control Logic and Implementing I/O Ports Example

The following figure shows the extraction of control logic and implementation of I/O ports for this code example:

```
void foo(int in[3], char a, char b, char c, int out[3]) {
    int x,y;
    for(int i = 0; i < 3; i++) {
        x = in[i];
        y = a*x + b + c;
        out[i] = y;
    }
}
```

Figure: Control Logic Extraction and I/O Port Implementation Example



X14219-100520

This code example performs the same operations as the previous example. However, it performs the operations inside a for-loop, and two of the function arguments are arrays. The resulting design executes the logic inside the for-loop three times when the code is scheduled. High-level synthesis automatically extracts the control logic from the C code and creates an FSM in the RTL design to sequence these operations. Top-level function arguments become ports in the final RTL design. The scalar variable of type `char` maps into a standard 8-bit data bus port. Array arguments, such as `in` and `out`, contain an entire collection of data. In high-level synthesis, arrays are synthesized into block RAM by default, but other options are possible, such as FIFOs, distributed RAM, and individual registers. When using arrays as arguments in the top-level function, high-level synthesis assumes that the block RAM is outside the top-level function and automatically creates ports to access a block RAM outside the design, such as data ports, address ports, and any required chip-enable or write-enable signals. The FSM controls when the registers store data and controls the state of any I/O control signals. The FSM starts in the state `C0`. On the next clock, it enters state `C1`, then state `C2`, and then state `C3`. It returns to state `C1` (and `C2`, `C3`) a total of three times before returning to state `C0`.

 **Note:** This closely resembles the control structure in the C code for-loop. The full sequence of states are: `C0`, `{C1, C2, C3}`, `{C1, C2, C3}`, `{C1, C2, C3}`, and return to `C0`.

The design requires the addition of `b` and `c` only one time. High-level synthesis moves the operation outside the for-loop and into state `C0`. Each time the design enters state `C3`, it reuses the result of the addition.

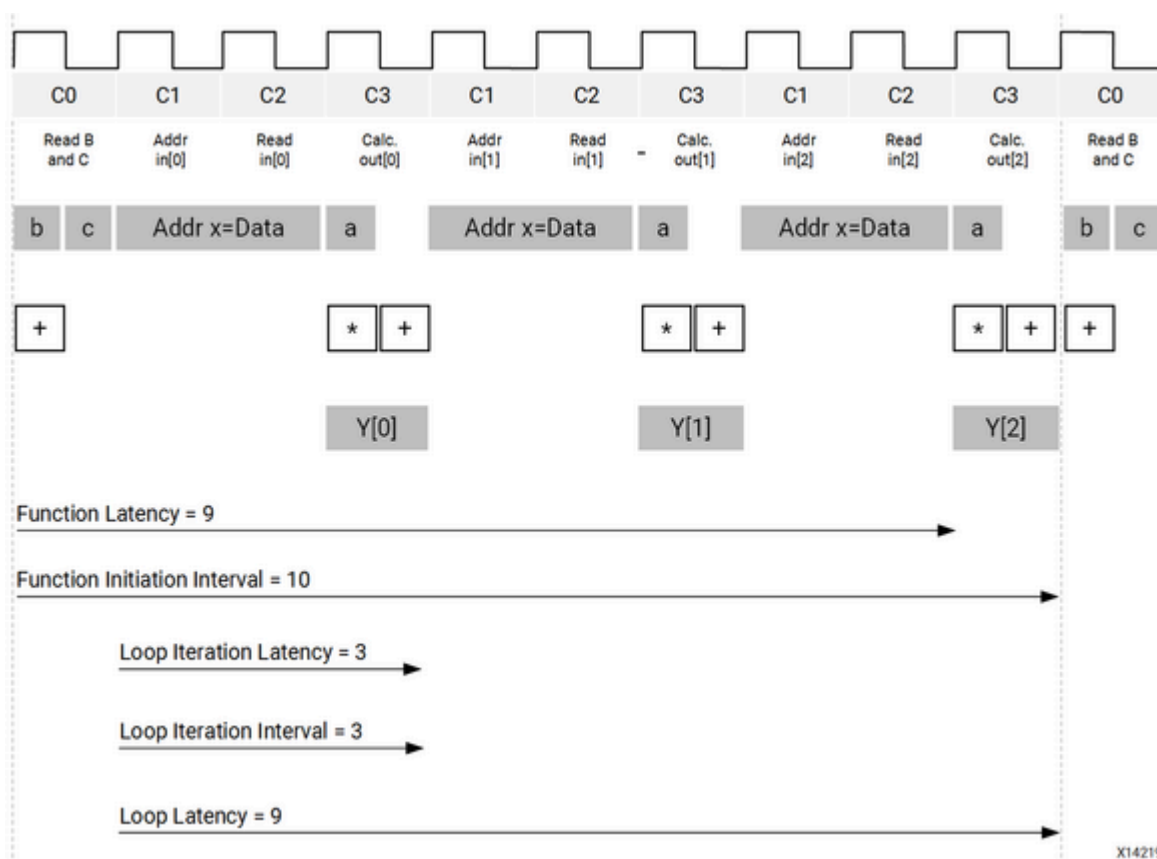
The design reads the data from `in` and stores the data in `x`. The FSM generates the address for the first element in state `C1`. In addition, in state `C1`, an adder increments to keep track of how many times the design must iterate around states `C1`, `C2`, and `C3`. In state `C2`, the block RAM returns the data for `in` and stores it as variable `x`.

High-level synthesis reads the data from port `a` with other values to perform the calculation and generates the first `y` output. The FSM ensures that the correct address and control signals are generated to store this value outside the block. The design then returns to state `C1` to read the next value from the array/block RAM `in`. This process continues until all outputs are written. The design then returns to state `C0` to read the next values of `b` and `c` to start the process again.

Performance Metrics Example

The following figure shows the complete cycle-by-cycle execution for the code in the previous example, including the states for each clock cycle, read operations, computation operations, and write operations.

Figure: Latency and Initiation Interval Example



The following are performance metrics for this example:

- Latency: It takes the function 9 clock cycles to output all values.

Note: When the output is an array, the latency is measured to the last array value output.

- Initiation Interval (II): The II is 10, which means it takes 10 clock cycles before the function can initiate a new set of input reads and start to process the next set of input data.

Note: The time to perform one complete execution of a function is referred to as one *transaction*. In this example, it takes 11 clock cycles before the function can accept data for the next transaction.

- Loop iteration latency: The latency of each loop iteration is 3 clock cycles.
- Loop II: The interval is 3.
- Loop latency: The latency is 9 clock cycles.

Optimizing Logic

Inferring Shift Registers

Vitis HLS will now infer a shift register when encountering the following code:

```
int A[N]; // This will be replaced by a shift register

for(...) {
    // The loop below is the shift operation
    for (int i = 0; i < N-1; ++i)
        A[i] = A[i+1];
    A[N] = ...;

    // This is an access to the shift register
    ... A[x] ...
}
```

Shift registers can perform a one shift operation per cycle, and also allows a random read access per cycle anywhere in the shift register, thus it is more flexible than a FIFO.

Optimizing Logic Expressions

During synthesis several optimizations, such as strength reduction and bit-width minimization are performed. Included in the list of automatic optimizations is expression balancing.

Expression balancing rearranges operators to construct a balanced tree and reduce latency.

- For integer operations expression balancing is on by default but may be disabled using the `EXPRESSION_BALANCE` pragma or directive.
- For floating-point operations, expression balancing is off by default but may be enabled using the `config_compile -unsafe_math_optimizations` command, as discussed below.

Given the highly sequential code using assignment operators such as `+=` and `*=` in the following example (or resulting from loop unrolling):

```
data_t foo_top (data_t a, data_t b, data_t c, data_t d)
{
```

```

data_t sum;

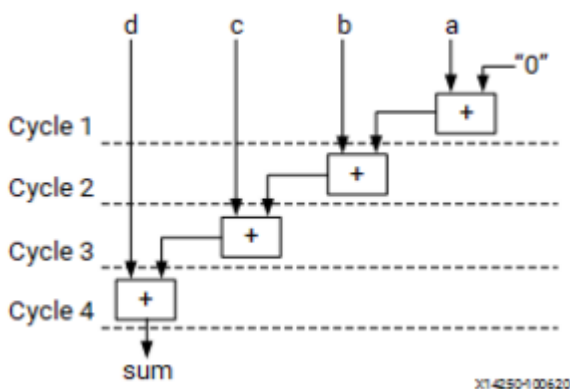
sum = 0;
sum += a;
sum += b;
sum += c;
sum += d;
return sum;

}

```

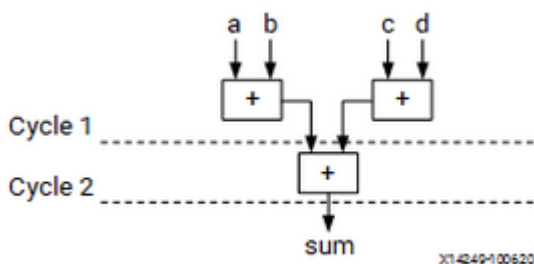
Without expression balancing, and assuming each addition requires one clock cycle, the complete computation for sum requires four clock cycles shown in the following figure.

Figure: Adder Tree



However additions $a+b$ and $c+d$ can be executed in parallel allowing the latency to be reduced. After balancing the computation completes in two clock cycles as shown in the following figure. Expression balancing prohibits sharing and results in increased area.

Figure: Adder Tree After Balancing



For integers, you can disable expression balancing using the `EXPRESSION_BALANCE` optimization directive with the `off` option. By default, the

HLS tool does not perform the `EXPRESSION_BALANCE` optimization for operations of type `float` or `double`. When synthesizing `float` and `double` types, the tool maintains the order of operations performed in the C/C++ code to ensure that the results are the same as the C/C++ simulation. For example, in the following code example, all variables are of type `float` or `double`. The values of `O1` and `O2` are not the same even though they appear to perform the same basic calculation.

```
A=B*C; A=B*F;
D=E*F; D=E*C;
O1=A*D O2=A*D;
```

This behavior is a function of the saturation and rounding in the C/C++ standard when performing operation with types `float` or `double`. Therefore, the HLS compiler always maintains the exact order of operations when variables of type `float` or `double` are present and does not perform expression balancing by default.

You can enable expression balancing for specific operations, or you can configure the tool to enable expression balancing with `float` and `double` types using the `syn.compile.unsafe_math_optimizations=1` configuration command.

With this setting enabled, the tool might change the order of operations to produce a more optimal design. However, the results of C/RTL co-simulation might differ from the C/C++ simulation.

The `unsafe_math_optimizations` feature also enables the `syn.compile.no_signed_zeros` optimization. The `no_signed_zeros` optimization ensures that the following expressions used with `float` and `double` types are identical:

```
x - 0.0 = x;
x + 0.0 = x;
0.0 - x = -x;
x - x = 0.0;
x*0.0 = 0.0;
```

Without the `no_signed_zeros` optimization the expressions above would not be equivalent due to rounding. The optimization may be optionally used without expression balancing by selecting only the `syn.compile.no_signed_zeros` configuration command.

★ **Tip:** When the `unsafe_math_optimizations` and `no_signed_zero` optimizations are used, the RTL implementation will have different results than the

C/C++ simulation. The test bench should be capable of ignoring minor differences in the result: check for a range, do not perform an exact comparison.

Optimizing AXI System Performance

Introduction

A Vitis accelerated system includes a global memory subsystem that is used to share data between the kernels and the host application. Global memory available on the host system, outside of the AMD device, provides very large amounts of storage space but at the cost of longer access time compared to local memory on the AMD device. One of the measurements of the performance of a system/application is throughput, which is defined as the number of bytes transferred in a given time frame. Therefore, inefficient data transfers from/to the global memory will have a long memory access time which can adversely affect system performance and kernel execution time.

Development of accelerated applications in Vitis HLS should include two phases: kernel development, and improving system performance. Design Principles suggested a kernel development approach implementing a cache-like Load-Compute-Store structure where the load-store functions read/write data to the global memory. Improving system performance involves implementing an efficient load and store design that can improve the kernel execution time. This chapter describes the features and metrics that can impact and improve the throughput of the load-store (LS) functions. Refer to [Vitis-HLS-Introductory-Examples/Interface/Memory](#) on GitHub for examples of some of the following concepts.

AXI Burst Transfers

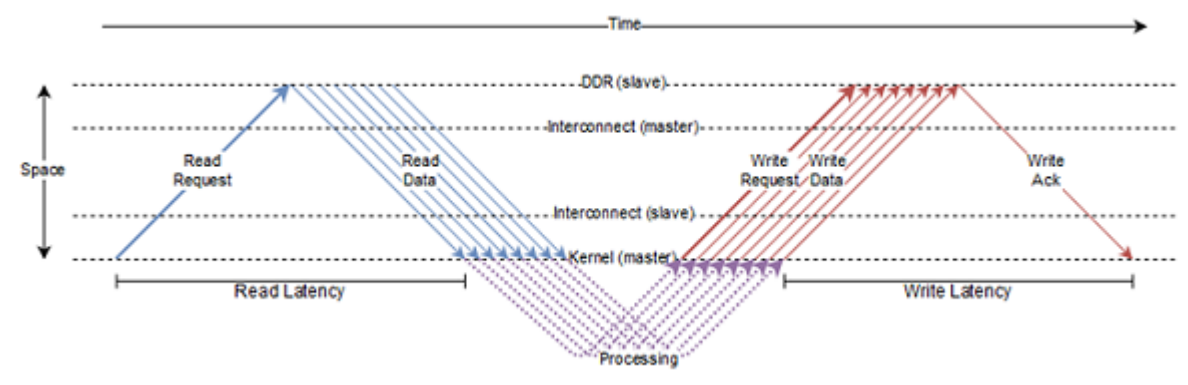
Overview of Burst Transfers

Bursting is an optimization that tries to intelligently aggregate your memory accesses to the DDR to maximize the throughput bandwidth and/or minimize the latency. Bursting is one of many possible optimizations to the kernel. Bursting typically gives you a 4-5x improvement while other optimizations, like access widening or ensuring there are no dependencies through the DDR memory, can provide even bigger performance improvements. Typically, bursting is useful when you have contention on the DDR ports from multiple competing kernels.

The Vitis HLS tool supports automatic burst inference, and supports a number of features such as automatic port-widening to facilitate automatic burst access. In some cases, where automatic burst access has failed, an efficient solution is to re-write the code or use manual burst as described in [Using Manual Burst](#). If that does not work, then another solution might be to use cache memory in the AXI4 interface using the `CACHE` pragma or directive.

The burst feature of the AXI4 protocol improves the throughput of the load-store functions by reading/writing chunks of data to or from the global memory in a single request. The larger the size of the data, the higher the throughput. This metric is calculated as follows $((\text{\#of bytes transferred}) * (\text{kernel frequency})) / (\text{Time})$. The maximum kernel interface bitwidth is 512 bits, and if the kernel is compiled to run at 300 MHz then it can theoretically achieve $(512 * 300 \text{ MHz}) / 1 \text{ sec} = \sim 17 \text{ GBps}$ for a DDR memory.

Figure: AXI Protocol



The preceding figure shows how the AXI protocol works. The HLS kernel sends out a read request for a burst of length 8 and then sends a write request burst of length 8. The read latency is defined as the time taken between the sending of the read request burst to when the data from the first read request in the burst is received by the kernel. Similarly, the write latency is defined as the time taken between when data for the last write in the write burst is sent and the time the write acknowledgment is received by the kernel. Read requests are usually sent at the first available opportunity while write requests get queued until the data for each write in the burst becomes available.

★ **Tip:** This behavior is determined by the `syn.interface.m_axi_conservative_mode` option as described in [Interface Configuration](#), which is enabled as the default behavior.

To understand the underlying semantics of burst transfers consider the following code snippet:

```
for(size_t i = 0; i < size; i++) {
    out[f(i)] = in[f(i)];
}
```

Vitis HLS performs automatic burst optimization, which intelligently aggregates the memory accesses inside the loops/functions from the user code and performs read/write to the global memory of a particular size. These read/writes are converted into a read request, write request, and write response to the global memory. Depending on the memory access pattern Vitis HLS automatically inserts these read and write requests either outside the loop bound or inside the loop body. Depending on the placement of these requests, Vitis HLS defines two types of burst requests: sequential burst and pipelined burst.

Burst Semantics

For a given kernel, the HLS compiler implements the burst analysis optimization as a multi-pass optimization, but on a per function basis. Bursting is only done for a function and bursting across functions is not supported. The burst optimizations are reported in the Synthesis Summary report, and missed burst opportunities are also reported to help you improve burst optimization.

At first, the HLS compiler looks for memory accesses in the basic blocks of the function, such as memory accesses in a sequential set of statements inside the function. Assuming the preconditions of bursting are met, each burst inferred in these basic blocks is referred to as *sequential* burst. The compiler will automatically scan the basic block to build the longest sequence of accesses into a single sequential burst.

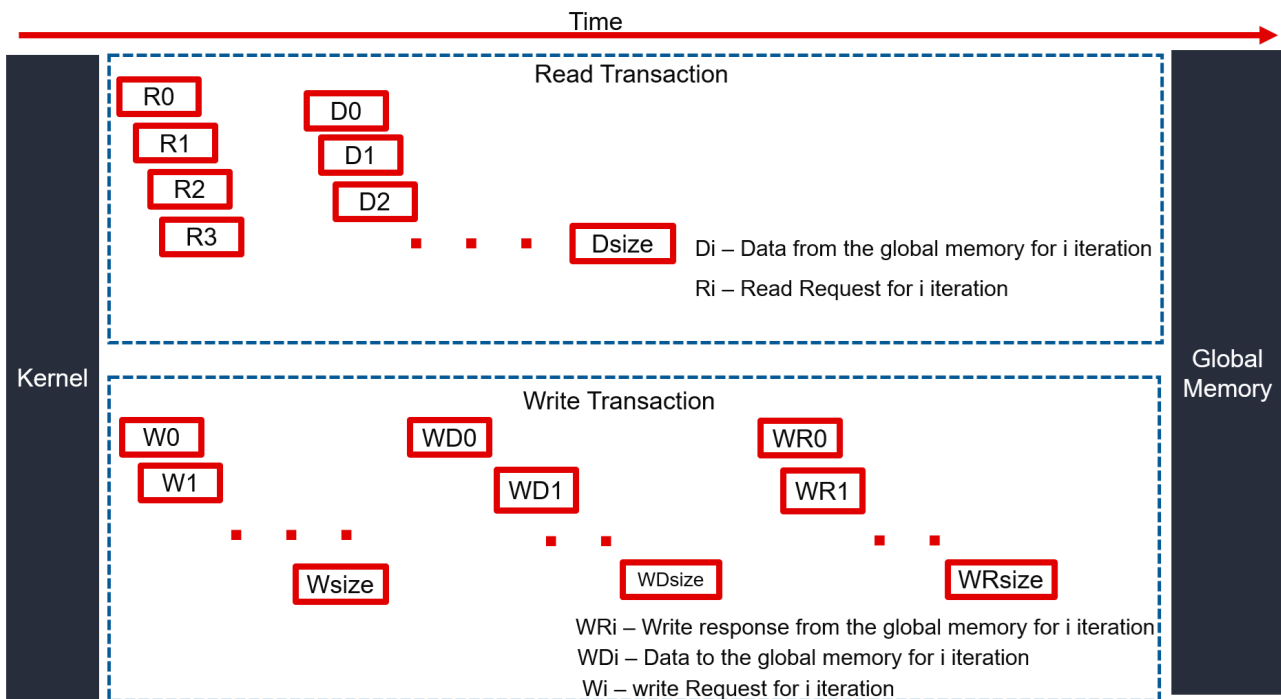
The compiler then looks at loops and tries to infer what are known as *pipeline* bursts. A pipeline burst is the sequence of reads/writes across the iterations of a loop. The compiler tries to infer the length of the burst by analyzing the loop induction variable and the trip count of the loop. If the analysis is successful, the compiler can chain the sequences of reads/writes in each iteration of the loop into one long pipeline burst. The compiler today automatically infers a pipeline or a sequential burst, but there is no way to request a specific type of burst. The code needs to be written so as to cause the tool to infer the pipeline or sequential burst.

Pipeline Burst

Pipeline burst improves the throughput of the functions by reading or writing large amounts, or the maximum amount of data in a single request. The advantage of the pipeline burst is that the future requests ($i+1$) do not have to wait for the current request (i) to finish because the read request, write request, and write response are outside the loop body and performs the requests as soon as possible, as shown in the following code example. This significantly improves the throughput of the functions as it takes less time to read/write the whole loop bound.

```
rb = ReadReq(i, size);
wb = WriteReq(i, size);
for(size_t i = 0; i < size; i++) {
    Write(wb, i) = f(Read(rb, i));
}
WriteResp(wb);
```

Figure: Pipeline Burst



If the compiler can successfully deduce the burst length from the induction variable (i) and the trip count of the loop ($size$), it will infer one big pipeline burst and will move the `ReadReq`, `WriteReq` and `WriteResp` calls outside the loop, as shown in the Pipeline Burst code example. So, the read requests for all loop iterations are combined into one read request and all the write requests are combined into one

write request. All read requests are typically sent out immediately while write requests are only sent out after the data becomes available.

In this case, the read and write requests for each loop iteration are combined into one read or write request. However, if any of the preconditions of bursting are not met, as described in [Preconditions and Limitations of Burst Transfer](#), the compiler might not infer a pipeline burst but will instead try and infer a sequential burst where the ReadReq, WriteReq and WriteResp are alongside the read/write accesses being burst optimized, as shown in the Sequential Burst code example.

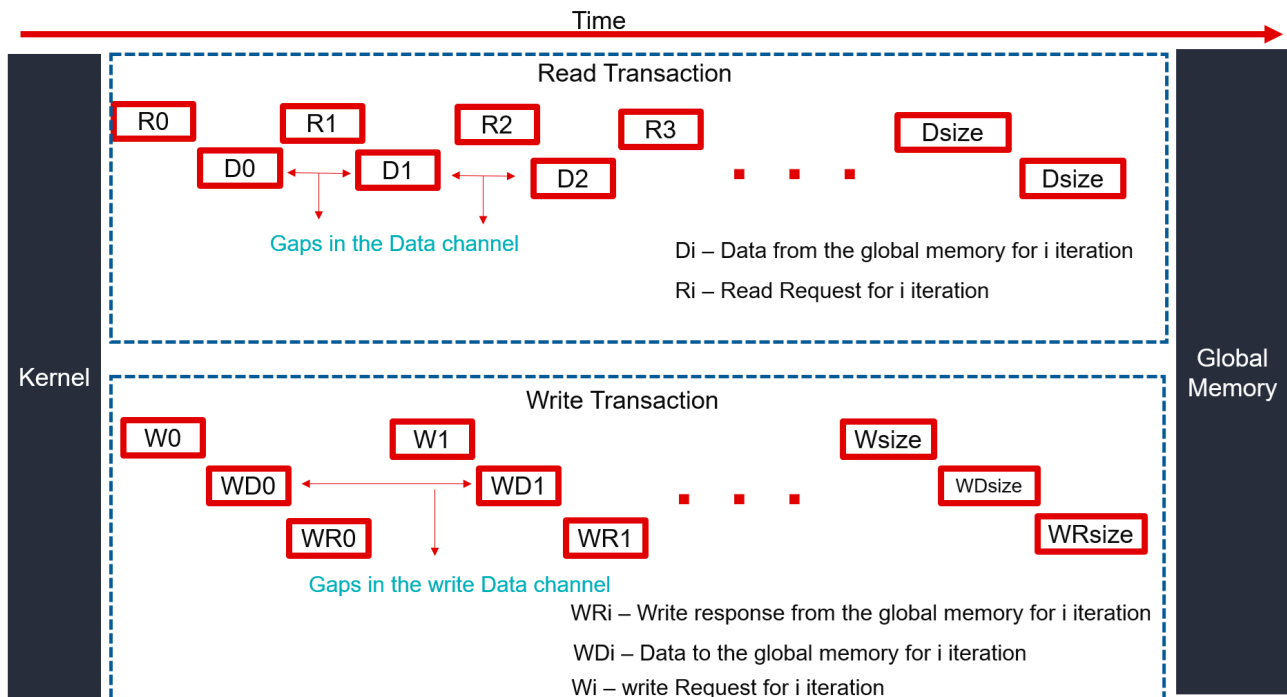
Sequential Burst

A sequential burst consists of smaller data sizes where the read requests, write requests, and write responses are inside a loop body as shown in the following code example.

```
for(size_t i = 0; i < size; i++) {
    rb = ReadReq(i, 1);
    wb = WriteReq(i, 1);
    Write(wb, i) = f(Read(rb, i));
    WriteResp(wb);
}
```

The drawback of sequential burst is that a future request (i+1) depends on the current request (i) finishing because it is waiting for the read request, write request, and write response to complete. This will create gaps between requests as shown in the following figure.

Figure: Sequential Burst



A sequential burst is not as effective as pipeline burst because it is reading or writing a small data size multiple times to compensate for the loop bounds. Although this will have a significant impact on the throughput, sequential burst is still better than no burst. Vitis HLS uses this burst technique if your code does not adhere to the [Preconditions and Limitations of Burst Transfer](#).

★ **Tip:** The size of burst requests can be further partitioned into multiple requests of user-specified size, which is controlled using the `max_read_burst_length` and `max_write_burst_length` of the `INTERFACE` pragma or directive, as discussed in [Options for Controlling AXI4 Burst Behavior](#).

Preconditions and Limitations of Burst Transfer

Bursting Preconditions

Bursting is about aggregating successive memory access requests. Here are the set of preconditions that these successive accesses must meet for the bursting optimization to launch successfully:

- Must be all reads, or all writes – bursting reads and writes is not possible.
- Must be a monotonically increasing order of access (both in terms of the memory location being accessed as well as in time). You cannot access a memory location that is in between two previously accessed memory locations.
- Must be consecutive in memory – one next to another with no gaps or overlap and in forward order.
- The number of read/write accesses (or burst length) must be determinable before the request is sent out. This means that even if the burst length is computed at runtime, it must be computed before the read/write request is sent out.
- If bundling two arrays to the same M-AXI port, bursting will be done only for one array, at most, in each direction at any given time.
- If same direction accesses on the same channel of the same bundle in the same region, there will be no burst for all these accesses.
- There must be no dependency issues from the time a burst request is initiated and finished.

★ **Tip:** The `volatile` qualifier prevents burst access to or from the variable.

Outer Loop Burst Failure Due to Overlapping Memory Accesses

Outer loop burst inference will fail in the following example because both iteration 0 and iteration 1 of the loop L1 access the same element in arrays a and b. Burst inference is an all or nothing type of optimization - the tool will not infer a partial burst. It is a greedy algorithm that tries to maximize the length of the burst. The auto-burst inference will try to infer a burst in a bottom up fashion - from the inner loop to the outer loop, and will stop when one of the preconditions is not met. In the example below the burst inference will stop when it sees that element 8 is being read again, and so an inner loop burst of length 9 will be inferred in this case.

```
L1: for (int i = 0; i < 8; ++i)
    L2: for (int j = 0; j < 9; ++j)
        b[i*8 + j] = a[i*8 + j];
```

```
itr 0: | 0 1 2 3 4 5 6 7 8 |
itr 1: |                   8 9 10 11 12 13 14 15 16 |
```

Usage of ap_int/ap_uint Types as Loop Induction Variables

Because the burst inference depends on the loop induction variable and the trip count, using non-native types can hinder the optimization from firing. It is recommended to always use unsigned integer type for the loop induction variable.

Must Enter Loop at Least Once

In some cases, the compiler can fail to infer that the max value of the loop induction variable can never be zero – that is, if it cannot prove that the loop will always be entered. In such cases, an assert statement will help the compiler infer this.

```
assert (N > 0);  
L1: for(int a = 0; a < N; ++a) { ... }
```

Inter or Intra Loop Dependencies on Arrays

If you write to an array location and then read from it in the same iteration or the next, this type of array dependency can be hard for the optimization to decipher. Basically, the optimization will fail for these cases because it cannot guarantee that the write will happen before the read.

Conditional Access to Memory

If the memory accesses are being made conditionally, it can cause the burst inferencing algorithm to fail as it cannot reason through the conditional statements. In some cases, the compiler will simplify the conditional and even remove it but it is generally recommended to not use conditional statements around the memory accesses.

M-AXI Accesses Made from Inside a Function Called from a Loop

Cross-functional array access analysis is not a strong suit for compiler transformations such as burst inferencing. In such cases, users can inline the

function using the `INLINE` pragma or directive to avoid burst failures.

```
void my_function(hls::stream<T> &out_pkt, int *din, int input_idx)
{
    T v;
    v.data = din[input_idx];
    out_pkt.write(v);
}

void my_kernel(hls::stream<T> &out_pkt,
               int          *din,
               int          num_512_bytes,
               int          num_times) {
#pragma HLS INTERFACE mode=m_axi port = din offset=slave
bundle=gmem0
#pragma HLS INTERFACE mode=axis port=out_pkt
#pragma HLS INTERFACE mode=s_axilite port=din bundle=control
#pragma HLS INTERFACE mode=s_axilite port=num_512_bytes
bundle=control
#pragma HLS INTERFACE mode=s_axilite port=num_times bundle=control
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    unsigned int idx = 0;
L0: for (int i = 0; i < ntimes; ++i) {
    L1: for (int j = 0; j < num_512_bytes; ++j) {
#pragma HLS PIPELINE
        my_function(out_pkt, din, idx++);
    }
}
}
```

Burst inferencing will fail because the memory accesses are being made from a called function. For the burst inferencing to work, it is recommended that users inline any such functions that are making accesses to the M-AXI memory.

An additional reason the burst inferencing will fail in this example is that the memory accessed through `din` in `my_function`, is defined by a variable (`idx`) which is not a function of the loop induction variables `i` and `j`, and therefore might not be sequential or monotonic. Instead of passing `idx`, use `(i*num_512_bytes+j)`.

Pipelined Burst Inference on a Dataflow Loop

Burst inference is not supported on a loop that has the DATAFLOW pragma or directive. However, each process/task inside the dataflow loop can have bursts. Also, sharing of M-AXI ports is not supported inside a dataflow region because the tasks can execute in parallel.

Options for Controlling AXI4 Burst Behavior

An optimal AXI4 interface is one in which the design never stalls while waiting to access the bus, and after bus access is granted, the bus never stalls while waiting for the design to read/write. There are many elements of the design that affect the system performance and burst transfer, such as the following:

- Latency
- Port Width
- Multiple Ports
- Specified Burst Length
- Number of Outstanding Reads/Writes

Latency

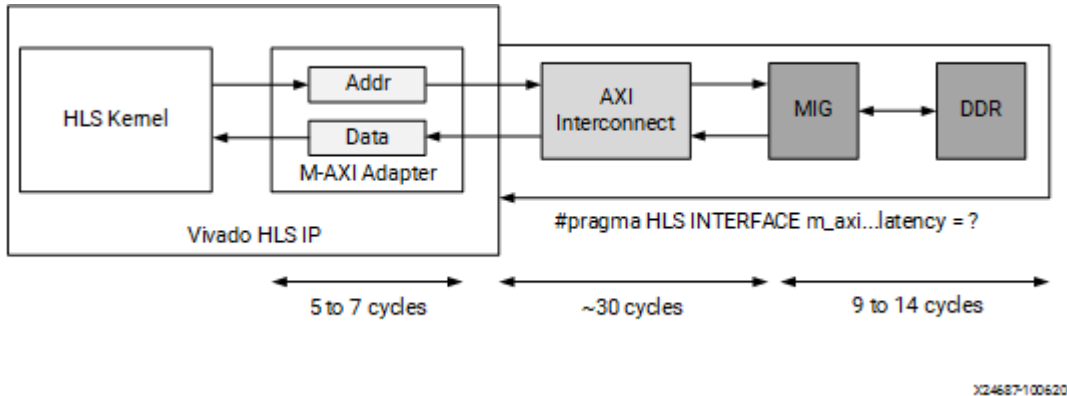
The read latency is defined as the time taken between sending the burst read request to when the kernel receives the data from the first read request in the burst. Similarly, the write latency is defined as the time it takes between when data for the last write in the burst is sent and the time the write response is received by the kernel. These latencies can be non-deterministic because they depend on system characteristics such as congestion on the DDR memory access. Because of this the HLS compiler can not accurately determine the memory read/write latency during synthesis, and so uses a default latency of 64 kernel cycles to schedule the requests and operations as below.

- It schedules the read/write requests and waits for the data, in parallel perform memory-independent operations, such as working on streams or compute
- Wait to schedule new read/write requests

★ **Tip:** The default tool latency can be changed using the LATENCY pragma or directive. The default latency of a specific interface can be changed using the latency option of the INTERFACE pragma or directive.

To help you understand the various latencies that are possible in the system, the following figure shows what happens when an HLS kernel sends a burst to the DDR memory.

Figure: Burst Transaction Diagram



When your design makes a read/write request, the request is sent to the DDR memory through several specialized helper modules. First, the `m_axi` adapter serves as a buffer for the requests created by the HLS kernel. The adapter contains logic to cut large bursts into smaller ones (which it needs to do to prevent hogging the channel or if the request crosses the 4 KB boundary, see *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))).

By default the adapter stalls the sending of burst requests until all data is available (depending on the maximum outstanding requests parameter) so that it can safely buffer the entirety of the data for each kernel. This is done to reduce deadlock due to concurrent requests (read or write) on the memory subsystem, but can increase write latency. You can disable this hold of write requests by setting `syn.interface.m_axi_conservative_mode=false` as described in Interface Configuration, but increase the risk of deadlock.

Getting through the adapter will cost a few cycles of latency, typically 5 to 7 cycles. Then, the request goes to the AXI interconnect that routes the kernel's request to the MIG and then eventually to the DDR memory. Getting through the interconnect is expensive in latency and can take around 30 cycles. Finally, getting to the DDR and back can cost anywhere from 9 to 14 cycles. These are not precise measurements of latency but rather estimates provided to show the relative latency cost of these specialized modules. For more precise measurements, you can test and observe these latencies using the Application Timeline report for your specific system, as described in [AXI Performance Case Study](#).

★ **Tip:** For information about the Application Timeline report, see [Timeline Trace](#) in the *Data Center Acceleration using Vitis* ([UG1700](#)).

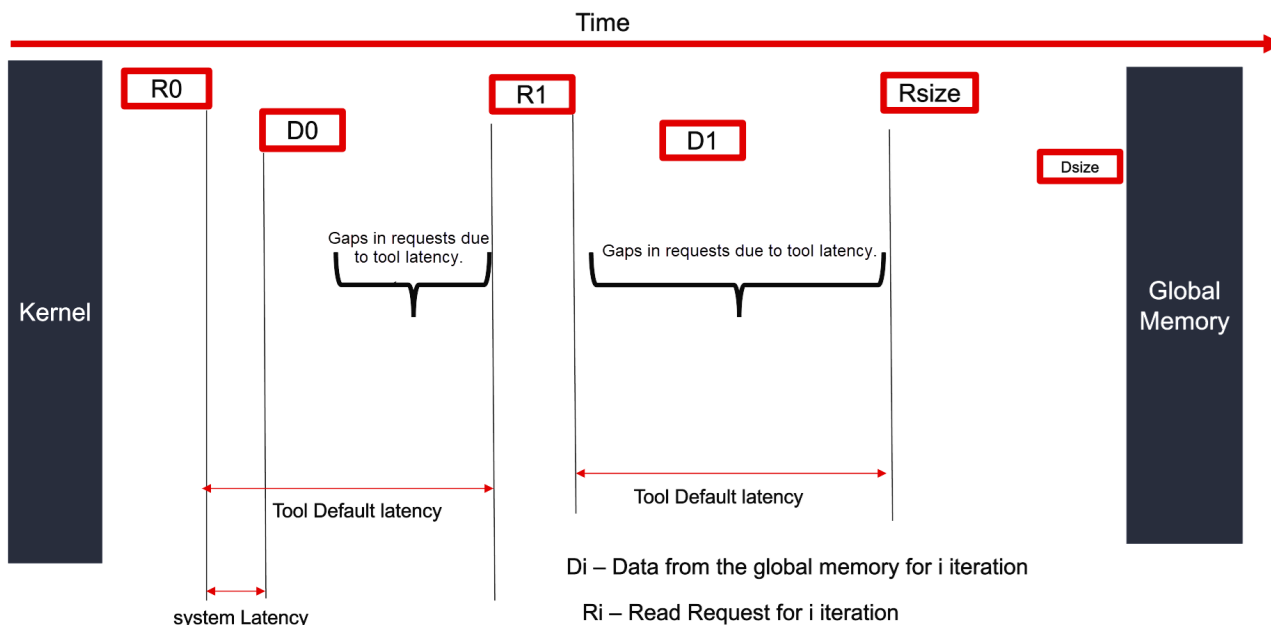
Another way to view the latencies in the system is as follows: the interconnect has an average II of 2 while the DDR memory controller has an average II of 4-5 cycles on requests (while on the data they are both II=1). The interconnect arbitration strategy is based on the size of read/write requests, and so data requested with longer burst lengths get prioritized over requests with shorter bursts (thus leading to a bigger channel bandwidth being allocated to longer bursts in case of contention). Of course, a large burst request has the side-effect of preventing anyone else from accessing the DDR memory, and therefore there must be a compromise between burst length and reducing DDR port contention. Fortunately, the large latencies help prevent some of this port contention, and effective pipelining of the requests can significantly improve the bandwidth throughput available in the system.

Latency does not affect loops/functions with pipelined bursts because the burst requests the maximum size in a single request.

Latency effects loops/functions with sequential burst in two possible ways:

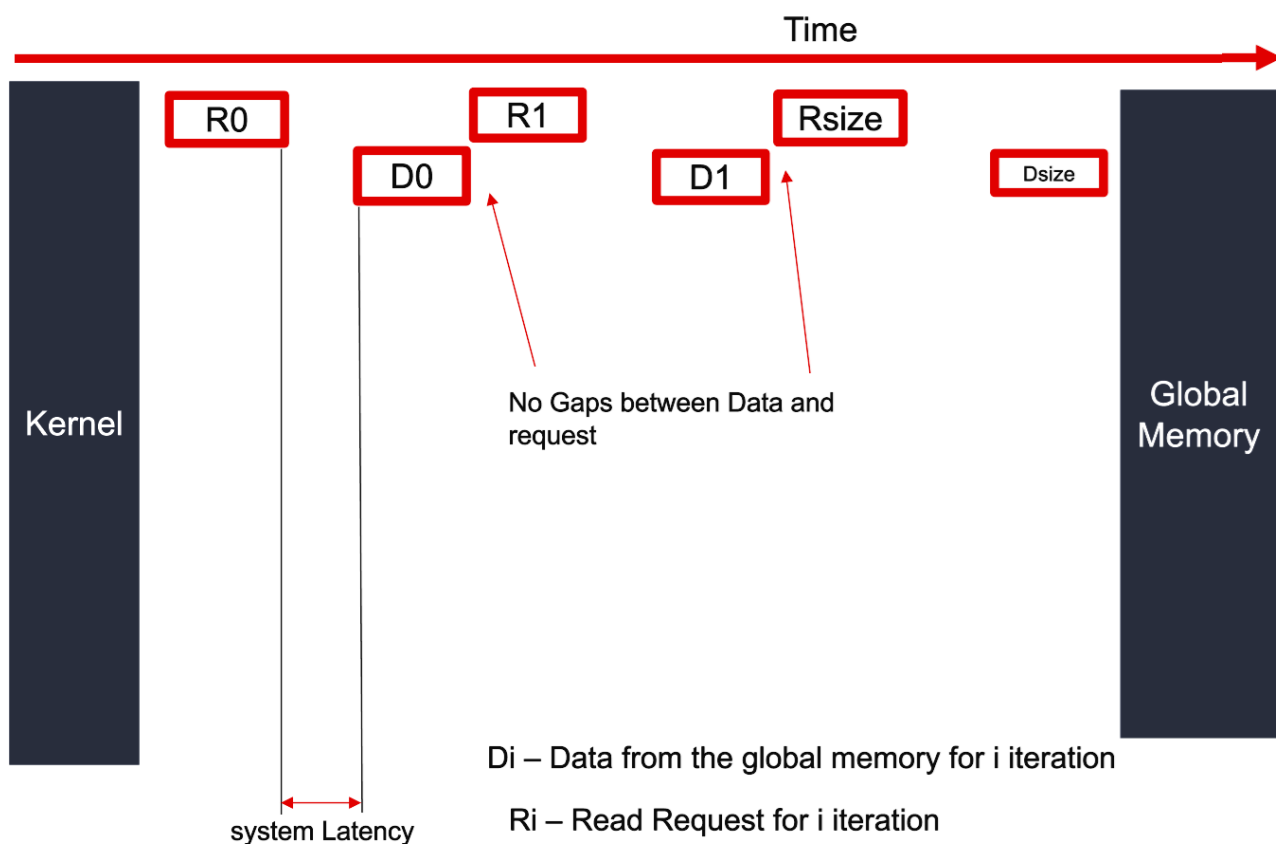
- If the system read/write latency is larger than the default tool latency, Vitis HLS has to wait for the data. Changing the latency will not improve the performance of the system.
- If the read/write latency is less than the tool default, then Vitis HLS sits in an idle state and wastes the remaining kernel cycles. This can impact the performance of the design because during this idle state it does not perform tasks. As you can see from the following figure the difference between the system latency and the default latency parameter will cause the sequential requests to be delayed further in time. This causes a significant loss of throughput.

Figure: Default Tool Latency



However, when you reduce the tool latency using the LATENCY pragma or directive, the tool will tightly pack the requests for a sequential burst, as shown in the following figure.

Figure: Adjusted Tool Latency



Recommended: Latency has a significant impact on sequential burst. Decreasing the default tool latency can improve the system performance.

Port Width

The throughput of load-store functions can be further improved by maximizing the number of bytes transferred. The Vitis unified IDE and v++ supports kernel ports up to 1024 bits wide, which means that a kernel can read or write up to 128 bytes per clock cycle per port.

 **Recommended:** You should maximize the port width of the interface, i.e., the bit-width of each AXI port, by setting it to 512 bits (64 bytes).

Vitis HLS also supports automatic port width optimization by analyzing the memory access pattern of the source code. If the code satisfies the preconditions and limitations for burst access, it will automatically resize the port to 512 bit width in the Vitis kernel flow.

!! Important: If the size and number of iterations are variable at compile time, then the tool will not automatically widen port widths.

If the tool cannot automatically widen the port, you can manually change the port width by using Vector Data Types or Arbitrary Precision (AP) Data Types as the data type of the port.

Multiple Ports and Channels

The throughput of load-store functions can be further improved by maximizing concurrent read/writes. In Vitis HLS, the function arguments by default are bundled/mapped/grouped to a single port. Bundling ports into a single port helps save resources. However, a single port can limit the performance of the kernel because all the memory transfers have to go through a single port. The `m_axi` interface has independent READ and WRITE channels, so a single port can read and write simultaneously. The `m_axi` Interface also provides channels in a single port, as described in [M_AXI Channels](#) to increase read and write channels in the port.

Using multiple ports lets you increase the bandwidth and throughput of the kernel by creating multiple interfaces to connect to different memory banks, as shown in the [Multi-DDR tutorial](#), or the accesses will be sequential. When multiple arguments are accessing the same memory port or memory bank, an arbiter will sequence the concurrent accesses to the same memory port or bank. Having multiple ports connected to different memory banks increases the throughput of the load and store functions, and as a result, the compute block should also be equally scaled to

meet the throughput demand from the load and store functions otherwise it will put back-pressure or stalls on the load-store functions.

 **Recommended:** Analyze the concurrent memory reads/writes and have a dedicated/independent port for concurrent accesses.

Number of Outstanding Reads/Writes

The throughput of load-store functions can be further improved by allowing the system to hide some of the memory latency. The `m_axi_num_read_outstanding` and `m_axi_num_write_outstanding` options of the `config_interface` command, or of the `INTERFACE` pragma or directive, lets the Kernel control the number of pipelined memory requests sent to the global memory without waiting for the previous request to complete.

Increasing the number of pipelined requests increases the pipeline depth of the read/write requests, which will cost additional BRAM/URAM resources.

 **Note:** In most cases where burst length ≥ 16 , the default number of outstanding reads/writes should be sufficient. For a burst of size less than 16, AMD recommends doubling the size of the number of outstanding from the default of 16.

Defining Burst Attributes with the INTERFACE Pragma

To create the optimal AXI4 interface, the following command options are provided in the `INTERFACE` directive to specify the behavior of the bursts and optimize the efficiency of the AXI4 interface.

Note that some of these options can use internal storage to buffer data and this may have an impact on area and resources:

latency

Specifies the expected latency of the AXI4 interface, allowing the design to initiate a bus request several cycles (latency) before the read or write is expected. If this figure is too low, the design will be ready too soon and may stall waiting for the bus. If this figure is too high, bus access may be granted but the bus may stall waiting on the design to start the access. Default latency in Vitis HLS is 64.

max_read_burst_length

Specifies the maximum number of data values read during a burst transfer. Default value is 16.

num_read_outstanding

Specifies how many read requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design: a FIFO of size $\text{num_read_outstanding} * \text{max_read_burst_length} * \text{word_size}$. Default value is 16.

max_write_burst_length

Specifies the maximum number of data values written during a burst transfer. Default value is 16.

num_write_outstanding

Specifies how many write requests can be made to the AXI4 bus, without a response, before the design stalls. This implies internal storage in the design: a FIFO of size $\text{num_write_outstanding} * \text{max_write_burst_length} * \text{word_size}$. Default value is 16.

The following INTERFACE pragma example can be used to help explain these options:

```
#pragma HLS interface mode=m_axi port=input offset=slave
bundle=gmem0
depth=1024*1024*16/(512/8) latency=100 num_read_outstanding=32
num_write_outstanding=32
max_read_burst_length=16 max_write_burst_length=16
```

- The interface is specified as having a latency of 100. The HLS compiler seeks to schedule the request for burst access 100 clock cycles before the design is ready to access the AXI4 bus.
- To further improve bus efficiency, the options `num_write_outstanding` and `num_read_outstanding` ensure the design contains enough buffering to store up to 32 read and/or write bursts. Each request will require its own buffer. This allows the design to continue processing until the bus requests are serviced.
- Finally, the options `max_read_burst_length` and `max_write_burst_length` ensure the maximum burst size is 16 and that the AXI4 interface does not hold the bus for longer than this. The HLS tool will partition longer bursts according to the specified burst length, and report this condition with a message like the following:

```
Multiple burst reads of length 192 and bit width 128 in loop
'VITIS_LOOP_2'(/src/filter.cpp:247:21)has been inferred on
port 'mm_read'.
```

These burst requests might be further partitioned into multiple requests during RTL generation based on the `max_read_burst_length` settings.

Commands to Configure the Burst

These commands configure global settings for the tool to optimize the AXI4 interface for the system in which it will operate. The efficiency of the operation depends on these values being set accurately. The provided default values are conservative, and might require changing depending on the memory access profile of your design.

Table: Vitis HLS Controls

Vitis HLS Command	Value	Description
<code>syn.interface.m_axi_conservative_mode</code>	<code>conservative_mode</code> default=true	Delay M-AXI each write request until the associated write data are entirely available (typically, buffered into the adapter or already emitted). This can

Vitis HLS Command	Value	Description
		slightly increase write latency but can resolve deadlock due to concurrent requests (read or write) on the memory subsystem.
<code>syn.interface.m_axi_latency</code>	unit 0 is auto default=0 (for Vivado IP flow) default=64 (for Vitis Kernel flow)	Provide the scheduler with an expected latency for M-AXI accesses. Latency is the delay between a read request and the first read data, or between the last write data and the write response. Note that this number need not be exact, underestimation makes for a lower-latency schedule, but with longer dynamic stalls. The scheduler will account for the additional adapter latency and add a few cycles.
<code>syn.interface.m_axi_min_width</code>	unit default=8	Minimum bitwidth for M-AXI interfaces data channels. Must be a power-of-two between 8 and 1024. Note that this does not necessarily increase throughput if the actual accesses are smaller than the required interface.

Vitis HLS Command	Value	Description
<code>syn.interface.m_axi_max_width</code>	minimum bitwidth default=1024	Minimum bitwidth for M-AXI interfaces data channels. Must be a power-of-two between 8 and 1024. Note that this does decrease throughput if the actual accesses are bigger than the required interface as they will be split into a multi-cycle burst of accesses.
<code>syn.interface.m_axi_max_widen_bitwidth</code>	minimum default=0 (for Vivado IP flow) default=512 (for Vitis Kernel flow)	Allow the tool to automatically widen bursts on M-AXI interfaces up to the chosen bitwidth. Must be a power-of-two between 8 and 1024. Note that burst widening requires strong alignment properties (in addition to burst).
<code>syn.interface.m_axi_auto_m_axi_ports</code>	tool default=false	If the option is false, all the M-AXI interfaces that are not explicitly bundled will be bundled into a single common interface, thus minimizing resource usage (single adapter). If the option is true, all the M-AXI interfaces that are not explicitly bundled will be mapped

Vitis HLS Command	Value	Description
		into individual interfaces, thus increasing the resource usage (multiple adapters).
<code>syn.interface.m_axi_align</code>	<code>uint</code> <code>alignment_byte_size</code> default=1 (for Vivado IP flow) default=64 (for Vitis Kernel flow)	Assume top function pointers that are mapped to M-AXI interfaces are at least aligned to the provided width in byte (power of two). This can help automatic burst widening. Warning: behavior will be incorrect if the pointers are not actually aligned at runtime.
<code>syn.interface.m_axi_num_read_outstanding</code>	<code>uint</code> default=16	Default value for M-AXI <code>num_read_outstanding</code> interface parameter.
<code>syn.interface.m_axi_num_write_outstanding</code>	<code>uint</code> default=16	Default value for M-AXI <code>num_write_outstanding</code> interface parameter.
<code>syn.interface.m_axi_max_read_burst_length</code>	<code>uint</code> default=16	Default value for M-AXI <code>max_read_burst_length</code> interface parameter.
<code>syn.interface.m_axi_max_write_burst_length</code>	<code>uint</code> default=16	Default value for M-AXI <code>max_write_burst_length</code> interface parameter.

Stencil Optimizations

The stencil pattern algorithm is a computational pattern used extensively in the field of scientific computing, image processing, and numerical simulations.

- This algorithm updates the value of each element in an array by applying a predetermined pattern or "stencil," which dictates how to amalgamate the values of adjacent elements.
- For instance, within image processing contexts, a stencil computation could calculate a weighted average of a pixel's value with its immediate neighbors to achieve a blurring effect. Stencil computations are demanding in terms of memory bandwidth on hardware like FPGAs, as the non-sequential nature of stencil pixel locations necessitates numerous DDR memory reads, markedly prolonging computation times.
- To mitigate this, stencil pattern algorithms will be implemented using window and line buffer techniques. These techniques optimize data access patterns, reducing the need for multiple off-chip memory reads by caching relevant data in an on-chip memory.

These techniques can improve memory bandwidth utilization and increase the effectiveness of parallelization and pipelining.

Adapting algorithms to leverage line and window buffers can be time-intensive, requiring considerable code refactoring. Vitis HLS introduces stencil optimization/pragma, which can automatically implement the line buffer and window buffer and achieve the same performance.

```
for(int y=0; y<30; ++y)
{
    for(int x=0; x<1000; ++x)
    {
#pragma HLS pipeline II=1
#pragma HLS array_stencil variable=src

        // Apply 2D filter to the pixel window
        int sum = 0;
        for(int row=0; row<FILTER_V_SIZE; row++)
        {

            for(int col=0; col<FILTER_H_SIZE; col++)
            {
                unsigned char pixel;
                int xoffset = (x+col-(FILTER_H_SIZE/2));
                int yoffset = (y+row-(FILTER_V_SIZE/2));
                // Deal with boundary conditions : clamp pixels
                to 0 when outside of image
```

```

        if ( (xoffset<0) || (xoffset>=1000) ||
(yoffset<0) || (yoffset>=30) ) {
            pixel = 0;
        } else {
            pixel = src[yoffset][xoffset];
        }
        sum += pixel*coeffs[row][col];
    }
}

```

Examples of Recommended Coding Styles

As described in Synthesis Summary, Vitis HLS issues a report summarizing burst activities and also identifying burst failures. If bursts of variable lengths are done, then the report will mention that bursts of variable lengths were inferred. The compiler also provides burst messages that can be found in the compiler log, `vitis_hls.log`. These messages are issued before the scheduling step.

Simple Read/Write Burst Inference

The following example is the standard way of reading and writing to the DDR and inferring a read and write burst. The Vitis HLS compiler will report the following burst inferences for the example below:

```

INFO: [HLS 214-115] Burst read of variable length and bit width 32
has been inferred on port 'gmem'
INFO: [HLS 214-115] Burst write of variable length and bit width 32
has been inferred on port 'gmem' (./src/vadd.cpp:75:9).

```

The code for this example follows:

```

/***** BEGIN EXAMPLE *****/

#define DATA_SIZE 2048
// Define internal buffer max size
#define BURSTBUFFERSIZE 256

//TRIPCOUNT identifiers
const unsigned int c_min = 1;
const unsigned int c_max = BURSTBUFFERSIZE;

```

```

const unsigned int c_chunk_sz = DATA_SIZE;

extern "C" {
void vadd(int *a, int size, int inc_value) {
    // Map pointer a to AXI4-master interface for global memory
    access
    #pragma HLS INTERFACE mode=m_axi port=a offset=slave bundle=gmem
    max_read_burst_length=256 max_write_burst_length=256
    // We also need to map a and return to a bundled axilite slave
    interface
    #pragma HLS INTERFACE mode=s_axilite port=a bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=size bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=inc_value bundle=control
    #pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    int burstbuffer[BURSTBUFFERSIZE];

    // Per iteration of this loop perform BURSTBUFFERSIZE vector
    addition
    for (int i = 0; i < size; i += BURSTBUFFERSIZE) {
    #pragma HLS LOOP_TRIPCOUNT min=c_min*c_min
    max=c_chunk_sz*c_chunk_sz/(c_max*c_max)
        int chunk_size = BURSTBUFFERSIZE;
        //boundary checks
        if ((i + BURSTBUFFERSIZE) > size)
            chunk_size = size - i;

        // Use a for loop to create a burst access to memory
        // memcpy is not recommended
        // memcpy(burstbuffer, &a[i], chunk_size * sizeof(int));
        for (i=0; i < chunk_size; i++) {
            burstbuffer[i] = a[i];
        }

        // Calculate and write results to global memory, the sequential
        write in a for loop can be
        // inferred as a memory burst access
        calc_write:
        for (int j = 0; j < chunk_size; j++) {
            #pragma HLS LOOP_TRIPCOUNT min=c_size_max max=c_chunk_sz
            #pragma HLS PIPELINE II=1
            burstbuffer[j] = burstbuffer[j] + inc_value;
        }
    }
}

```

```

        a[i + j] = burstbuffer[j];
    }
}

```

Pipelining Between Bursts

The following example will infer bursts of length N:

```

for(int x=0; x < k; ++x) {
    int off = f(x);
    for(int i = 0; i < N; ++i) {
        #pragma HLS PIPELINE II=1
        ... = gmem[off + i];
    }
}

```

But notice that the outer loop is not pipelined. This means that while there is pipelining inside bursts, there won't be any pipelining between bursts.

To remedy this you can unroll the inner loop and pipeline the outer loop to get pipelining between bursts as well. The following example will still infer bursts of length N, but now there will also be pipelining between bursts leading to higher throughput:

```

for(int x=0; x < k; ++x) {
    #pragma HLS PIPELINE II=N
    int off = f(x);
    for(int i = 0; i < N; ++i) {
        #pragma HLS UNROLL
        ... = gmem[off + i];
    }
}

```

Accessing Row Data from a Two-Dimensional Array

The following is an example of reading/writing to/from a two dimensional array. Vitis HLS infers read and write bursts and issues the following messages:

INFO: [HLS 214-115] Burst read of length 256 and bit width 512 has been inferred on port 'gmem' (./src/row_array_2d.cpp:43:5)

INFO: [HLS 214-115] Burst write of length 256 and bit width 512 has been inferred on port 'gmem' (./src/row_array_2d.cpp:56:5)

Notice that a bit width of 512 is achieved in this example. This is more efficient than the 32 bit width achieved in the simple example above. Bursting wider bit widths is another way bursts can be optimized as discussed in Automatic Port Width Resizing.

The code for this example follows:

```

/***** BEGIN EXAMPLE *****/
// Parameters Description:
//          NUM_ROWS:          matrix height
//          WORD_PER_ROW:      number of words in a row
//          BLOCK_SIZE:        number of words in an array
#define NUM_ROWS    64
#define WORD_PER_ROW 64
#define BLOCK_SIZE (WORD_PER_ROW*NUM_ROWS)

// Default datatype is integer
typedef int DTYPE;
typedef hls::stream<DTYPE> my_data_fifo;

// Read data function: reads data from global memory
void read_data(DTYPE *inx, my_data_fifo &inFifo) {
read_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
read_loop_jj:
        for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
            #pragma HLS PIPELINE II=1
            inFifo << inx[WORD_PER_ROW * i + jj];
        }
    }
}

// Write data function - writes results to global memory
void write_data(DTYPE *outx, my_data_fifo &outFifo) {
write_loop_i:
    for (int i = 0; i < NUM_ROWS; ++i) {
write_loop_jj:

```

```

        for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
            #pragma HLS PIPELINE II=1
            outFifo >> outx[WORD_PER_ROW * i + jj];
        }
    }
}

// Compute function is pretty simple because this example is
// focused on efficient
// memory access pattern.
void compute(my_data_fifo &inFifo, my_data_fifo &outFifo, int
alpha) {
    compute_loop_i:
        for (int i = 0; i < NUM_ROWS; ++i) {
            compute_loop_jj:
                for (int jj = 0; jj < WORD_PER_ROW; ++jj) {
                    #pragma HLS PIPELINE II=1
                    DTYPE inTmp;
                    inFifo >> inTmp;
                    DTYPE outTmp = inTmp * alpha;
                    outFifo << outTmp;
                }
            }
        }

extern "C" {
    void row_array_2d(DTYPE *inx, DTYPE *outx, int alpha) {
        // AXI master interface
        #pragma HLS INTERFACE mode=m_axi port = inx offset = slave
        bundle = gmem
        #pragma HLS INTERFACE mode=m_axi port = outx offset = slave
        bundle = gmem
        // AXI slave interface
        #pragma HLS INTERFACE mode=s_axilite port = inx bundle =
        control
        #pragma HLS INTERFACE mode=s_axilite port = outx bundle =
        control
        #pragma HLS INTERFACE mode=s_axilite port = alpha bundle =
        control
        #pragma HLS INTERFACE mode=s_axilite port = return bundle =
        control
    }
}

```

```

    my_data_fifo inFifo;
    // By default the FIFO depth is 2, user can change the
depth by using
    // #pragma HLS stream variable=inFifo depth=256
    my_data_fifo outFifo;

    // Dataflow enables task level pipelining, allowing
functions and loops to execute
    // concurrently. For more details please refer to UG902.
#pragma HLS DATAFLOW
    // Read data from each row of 2D array
    read_data(inx, inFifo);
    // Do computation with the acquired data
    compute(inFifo, outFifo, alpha);
    // Write data to each row of 2D array
    write_data(outx, outFifo);
    return;
}
}

```

Summary

Write code in such a way that bursting can be inferred. Ensure that none of the preconditions are violated. Bursting does not mean that you will get all your data in one shot – it is about merging the requests together into one request, but the data will arrive sequentially, one after another.

Burst length of 16 is ideal, but even burst lengths of 8 are enough. Bigger bursts have more latency while shorter bursts can be pipelined. Do not confuse bursting with pipelining, but note that bursts can be pipelined with other bursts.

If your bursts are of fixed length, you can unroll the inner loop where bursts are inferred and pipeline the outer loop. This will achieve the same burst length, but also pipelining between the bursts to enable higher throughput.

For greater throughput, focus on widening the interface up to 1024 bits rather than simply achieving longer bursts.

Bigger bursts have higher priority with the AXI interconnect. No dynamic arbitration is done inside the kernel.

You can have two `m_axi` ports connected to same DDR memory to model mutually exclusive access inside kernel, but the AXI interconnect outside the kernel will arbitrate competing requests.

One way to get around the out-of-order access restriction is to create your own buffer in block RAM, store the bursts in this buffer and then use this buffer to do out of order accesses. This is typically called a line buffer and is a common optimization used in video processing.

As another alternative, you can consider adding the CACHE pragma or directive as described in pragma HLS cache to improve performance of the `m_axi` interface. Review the Burst Optimization section of the Synthesis Summary report to learn more about burst optimizations in the design, and missed burst opportunities. If automatic burst is not occurring in your design, you might want to use the `hls::burst_maxi` data type for manual burst, as described in [Using Manual Burst](#).

Using Manual Burst

Burst transfers improve the throughput of the I/O of the kernel by reading or writing large chunks of data to the global memory. The larger the size of the burst, the higher the throughput, this metric is calculated as follows $((\# \text{ of bytes transferred}) * (\text{kernel frequency}) / (\text{Time}))$. The maximum kernel interface bitwidth is 512 bits and if the kernel is compiled at 300 MHz, then it can theoretically achieve $= (80\text{-}95\% \text{ efficiency of the DDR}) * (512 * 300 \text{ MHz}) / 1 \text{ sec} = \sim 17\text{-}19 \text{ GB/s}$ for a DDR. As explained, Vitis HLS performs automatic burst optimizations which intelligently aggregates the memory accesses of the loops/functions from the user code and performs read/write of a particular size in a single burst request. However, burst transfer also has requirements that can sometimes be overburdening or difficult to meet, as discussed in [Preconditions and Limitations of Burst Transfer](#).

In some cases, where automatic burst access has failed, an efficient solution is to re-write the code or use manual burst. In such cases, if you are familiar with the AXI4 `m_axi` protocol, and understand hardware transaction modeling, you can implement manual burst transfers using the `hls::burst_maxi` class as described below.

Refer to [Vitis-HLS-Introductory-Examples/Interface/Memory/manual_burst](#) on GitHub for examples of these concepts. Another solution might be to use cache memory in the AXI4 interface using the CACHE pragma or directive.

hls::burst_maxi Class

The `hls::burst_maxi` class provides a mechanism to perform read/write access to the DDR memory. These methods will translate the class methods usage behavior into respective AXI4 protocol and send and receive requests on the AXI4 bus signals - AW, AR, WDATA, BVALID, RDATA. These methods control the burst behavior of the HLS scheduler. The adapter, which receives the commands from

the scheduler, is responsible for sending the data to the DDR memory. These requests will adhere to the user specified INTERFACE pragma options, such as `max_read_burst_length` and `max_write_burst_length`. The class methods should only be used in the kernel code, and not in the test bench (except for the class constructor as described below).

- Constructors:

- `burst_maxi(const burst_maxi &B) : Ptr(B.Ptr) {}`
- `burst_maxi(T *p) : Ptr(p) {}`

!! Important: The HLS design and test bench must be in different files, because the constructor `burst_maxi(T *p)` is only available in C-simulation model.

- Read Methods:

- `void read_request(size_t offset, size_t len);`

This method is used to perform a read request to the `m_axi` adapter. The function returns immediately if the read request queue inside `m_axi` adapter is not full, otherwise it waits until space becomes available.

- `offset`: Specify the memory offset from which to read the data
- `len`: Specify the scheduler burst length. This burst length is sent to the adapter, which can then convert it to the standard AXI AMBA protocol

- `T read();`

This method is used to transfer the data from the `m_axi` adapter to the scheduler FIFO. If the data is not available, `read()` will be blocking. The `read()` method should be called `len` number of times, as specified in the `read_request()`.

- Write Methods:

- `void write_request(size_t offset, size_t len);`

This method is used to perform a write request to the `m_axi` adapter. The function returns immediately if the write request queue inside `m_axi` adapter is not full.

- `offset`: Specify the memory offset into which the data should be written
- `len`: Specify the scheduler burst length. This burst length is sent to the adapter, which can then convert it to the standard AXI AMBA protocol

- `void write(const T &val, ap_int<sizeof(T)> byteenable_mask = -1);`

This method is used to transfer data from the internal buffer of the scheduler to the `m_axi` adapter. It blocks if the internal write buffer is full. The `byteenable_mask` is used to enable the bytes in the WDATA. By default it will enable all the bytes of the transfer. The `write()` method should be called `len` number of times, as specified in the `write_request()`.

- `void write_response();`

This method blocks until all write responses are back from the global memory. This method should be called the same number of times as `write_request()`.

Using Manual Burst in HLS Design

In the HLS design, when you find that automatic burst transfers are not working as desired, and you cannot optimize the design as needed, you can implement the read and write transactions using the `hls::burst_maxi` object. In this case, you will need to modify your code to replace the original pointer argument with `burst_maxi` as a function argument. These arguments must be accessed by the explicit read and write methods of the `burst_maxi` class, as shown in the following examples.

The following shows an original code sample, which uses a pointer argument to read data from global memory.

```

void dut(int *A) {
    for (int i = 0; i < 64; i++) {
        #pragma pipeline II=1
        ... = A[i]
    }
}

```

In the modified code below, the pointer is replaced with the `hls::burst_maxi<>` class objects and methods. In the example, the HLS scheduler puts 4 requests of len 16 from port A to the `m_axi` adapter. The Adapter stores them inside a FIFO and whenever the AW/AR bus is available it will send the request to the global memory. In the 64 loop iterations, the `read()` command issues a blocking call that will wait for the data to come back from the global memory. After the data becomes available the HLS scheduler will read it from the `m_axi` adapter FIFO.

```

#include "hls_burst_maxi.h"
void dut(hls::burst_maxi<int> A) {
    // Issue 4 burst requests
    A.read_request(0, 16); // request 16 elements, starting from A[0]
    A.read_request(128, 16); // request 16 elements, starting from
A[128]
    A.read_request(256, 16); // request 16 elements, starting from
A[256]
    A.read_request(384, 16); // request 16 elements, starting from
A[384]
    for (int i = 0; i < 64; i++) {
        #pragma pipeline II=1
        ... = A.read(); // Read the requested data
    }
}

```

In example 2 below, the HLS scheduler/kernel puts 2 requests from port A to the adapter, the first request of len 2, and the second request of len 1, for a total of 2 write requests. It then issues corresponding, because the total burst length is 3 write commands. The Adapter stores these requests inside a FIFO and whenever the AW, W bus is available it will send the request and data to the global memory. Finally, two `write_response` commands are used, to await response for the two `write_requests`.

```

void trf(hls::burst_maxi<int> A) {
    A.write_request(0, 2);

```

```

A.write(x); // write A[0]
A.write_request(10, 1);
A.write(x, 2); // write A[1] with byte enable 0010
A.write(x); // write A[10]
A.write_response(); // response of write_request(0, 2)
A.write_response(); // response of write_request(10, 1)
}

```

Using Manual Burst in C-Simulation

Manual burst supports `read_request()` and `read()` in different dataflow processes to enable the `read_request()` and `read()` in parallel (see example below). Similarly, support for `write_request()`, `write()` and `write_response()` is also available.

```

void foo(hls::burst_maxi<int> a) {
    for (int i = 0; i < 10; i++) {
        #pragma HLS pipeline
        a.read_request(i * 16, 10);
    }
}

void bar(hls::burst_maxi<int> a, int *b) {
    for (int i = 0; i < 100; i++) {
        #pragma HLS pipeline
        b[i] = a.read();
    }
}

void test(hls::burst_maxi<int> a, int b[1000]) {
    #pragma HLS interface m_axi port=a depth=1000
    #pragma HLS dataflow
    foo(a);
    bar(a, b);
}

```

You can pass a regular array to the top function, and the array will be transformed to `hls::burst_maxi` automatically by the constructor.

!! Important: The HLS design and test bench must be in different files, because the `burst_maxi(T *p)` constructor is only valid for use in C simulation model.

```
#include "hls_burst_maxi.h"

void dut(hls::burst_maxi<int> A);

int main() {
    int Array[1000];
    dut(Array);
    .....
}
```

Using Manual Burst to Optimize Performance

Vitis HLS characterizes two types of burst behaviors: pipeline burst, and sequential burst.

Pipeline Burst

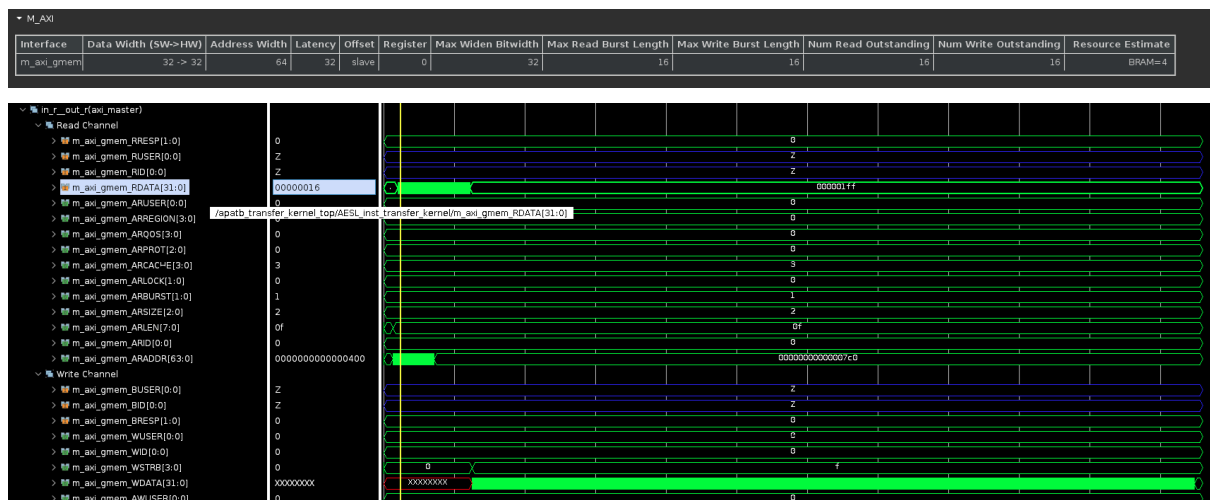
Pipeline Burst improves throughput by reading or writing the maximum amount of data in a single request. The compiler infers pipeline burst if the `read_request`, `write_request` and `write_response` calls are outside the loop, as shown in the following code example. In the below example the size is a variable that is sent from the test bench.

```
9  int buf[8192];
10  in.read_request(0, size);
11  for (int i = 0; i < size; i++) {
12      #pragma HLS PIPELINE II=1
13      buf[i] = in.read();
14      out.write_request(0, size*NT);
17      for (int i = 0; i < NT; i++) {
19          for (int j = 0; j < size; j++) {
20              #pragma HLS PIPELINE II=1
21              int a = buf[j];
22              out.write(a);
23          }
24      }
25  out.write_response();
```

Figure: Synthesis Results

M_AXI Burst Information					
Manual Burst (burst_maxi)					
HW Interface	Direction	Length	Width	Location	
m_axi_gmem	read	variable	32	example.cpp:10:12	
m_axi_gmem	write	variable	32	example.cpp:16:7	

As you can see from the preceding figure, the tool has inferred the burst from the user code and length is mentioned as variable at compile time.

Figure: Performance Benefits

During the runtime the HLS compiler sends a burst request of length = size and the adapter will partition them into the user-specified burst_length pragma option. In this case the default burst length is set to 16, which is used in the AR1en and AW1en channels. The read/write channel achieved maximum throughput because there are no bubbles during the transfer.

Figure: Co-sim Results

Modules & Loops	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
transfer_kernel				5735	5735	5735
transfer_kernel_Pipeline_VITIS_LOOP_11_1				517	517	517
transfer_kernel_Pipeline_VITIS_LOOP_18_2_VITIS_LOOP_19_3				5122	5122	5122

Sequential Burst

This burst is a sequential burst of smaller data sizes, where the read requests, write requests and write responses are inside the loop body as shown in the below snippet. The drawback of the sequential burst is that the future request ($i+1$) depends on the previous request (i) to finish because it is waiting for the read request, write request and write response to complete, this will cause gaps between requests. Sequential burst is not as effective as pipeline burst because it is reading or writing a small data size multiple times to compensate for the loop bounds. Although this will limit the improvement to throughput, sequential burst is still better than no burst.

```

void transfer_kernel(hls::burst_maxi<int>
in,hls::burst_maxi<int> out, const int size )
{
    #pragma HLS INTERFACE m_axi port=in depth=512 latency=32
offset=slave
    #pragma HLS INTERFACE m_axi port=out depth=5120 offset=slave
latency=32

    int buf[8192];

    for (int i = 0; i < size; i++) {
        in.read_request(i, 1);
    #pragma HLS PIPELINE II=1
        buf[i] = in.read();
    }

    for (int i = 0; i < NT; i++) {
        for (int j = 0; j < size; j++) {
            out.write_request(j, 1);
    #pragma HLS PIPELINE II=1
            int a = buf[j];
            out.write(a);
            out.write_response();

        }

    }

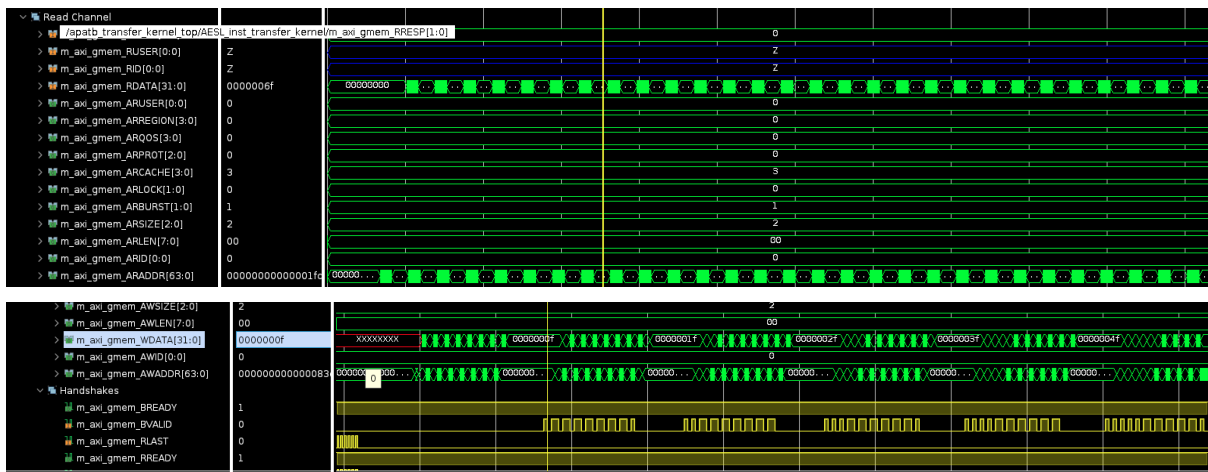
}

```

Figure: Synthesis Results

M_AXI Burst Information					
Manual Burst (burst_maxi)					
HW Interface	Direction	Length	Width	Location	
m_axi_gmem	read	1	32	example.cpp:13:8	
m_axi_gmem	write	1	32	example.cpp:23:15	

As you can see from the report sample above, the tool achieved a burst of length =1.

Figure: Performance Impacts

The read/write loop R/WDATA channel has gaps equal to read/write latency, as discussed in AXI4 Master Interface. For the read channel, the loop waits for all the read data to come back from the global memory. For the write channel, the innermost loop waits for the response (BVALID) to come back from the global memory. This results in performance degradation. The co-sim results also show that a 2x degradation in performance for this burst semantics.

Figure: Performance Estimates

Performance Estimates							
Module	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency	
transfer_kernel				13106	13106	13106	
transfer_kernel_Pipeline_VITIS_LOOP_12_1				1208	1208	1208	
transfer_kernel_Pipeline_VITIS_LOOP_20_2_VITIS_LOOP_22_3				11879	11879	11879	

Features and Limitations

1. If the `m_axi` element is a struct:
 - The struct will be packed into a wide int. Disaggregation of the struct is not allowed.
 - The size of struct must be a power-of-2, and should not exceed 1024 bits or the max width specified by the `config_interface - m_axi_max_bitwidth` command.
2. `ARRAY_PARTITION` and `ARRAY_RESHAPE` of `burst_maxi` ports is not allowed.
3. You can apply the `INTERFACE` pragma or directive to `hls::burst_maxi`, defining an `m_axi` interface. If the `burst_maxi` port is bundled with other ports, all ports in this bundle must be `hls::burst_maxi` and must have the same element type.

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B, int
*C, hls::burst_maxi<short> D) {
    #pragma HLS interface m_axi port=A offset=slave bundle=gmem
    // OK
    #pragma HLS interface m_axi port=B offset=slave bundle=gmem
    // OK
    #pragma HLS interface m_axi port=C offset=slave bundle=gmem
    // Bad. C must also be hls::burst_maxi type, because it shares
    the same bundle 'gmem' with A and B
    #pragma HLS interface m_axi port=D offset=slave bundle=gmem
    // Bad. D should have 'int' element type, because it shares
    the same bundle 'gmem' with A and B
}
```

4. You can use the `INTERFACE` pragma or directive to specify the `num_read_outstanding` and `num_write_outstanding`, and the `max_read_burst_length` and `max_write_burst_length` to define the size of the internal buffer of the `m_axi` adapter.

```
void dut(hls::burst_maxi<int> A) {
    #pragma HLS interface m_axi port=A num_read_outstanding=32
    num_write_outstanding=32 max_read_burst_length=16
    max_write_burst_length=16
}
```

5. The INTERFACE pragma or directive `max_widen_bitwidth` is not supported, because HLS will not change the bit width of `hls::burst_maxi` ports.
6. You must make a `read_request` before `read`, or `write_request` before `write`:

```
void dut(hls::burst_maxi<int> A) {
    ... = A.read(); // Bad because read() before read_request().
    You can catch this error in C-sim.
    A.read_request(0, 1);
}
```

7. If the address and life time of the read group (`read_request() > read()`) and write group (`write_request() > write() > write_response()`) overlap, the tool cannot guarantee the access order. C-simulation will report an error.

```
void dut(hls::burst_maxi<int> A) {
    A.write_request(0, 1);
    A.write(x);
    A.read_request(0, 1);
    ... = A.read(); // What value is read? It is undefined. It
    could be original A[0] or updated A[0].
    A.write_response();
}
```

```
void dut(hls::burst_maxi<int> A) {
    A.write_request(0, 1);
    A.write(x);
    A.write_response();
    A.read_request(0, 1);
    ... = A.read(); // this will read the updated A[0].
}
```

8. If multiple `hls::burst_maxi` ports are bundled to same `m_axi` adapter and their transaction lifetimes overlap, the behavior is unexpected.

```
void dut(hls::burst_maxi<int> A, hls::burst_maxi<int> B) {
    #pragma HLS INTERFACE m_axi port=A bundle=gmem depth = 10
    #pragma HLS INTERFACE m_axi port=B bundle=gmem depth = 10
    A.read_request(0, 10);
    B.read_request(0, 10);
}
```

```

    for (int i = 0; i < 10; i++) {
        #pragma HLS pipeline II=1
        ..... = A.read(); // get value of A[0], A[2], A[4] ...
        ..... = B.read(); // get value of A[1], A[3], A[5] ...
    }
}

```

9. Read or write requests and read or writes in different dataflow process are not supported. Dataflow checker will report an error: multiple writes in different dataflow processes are not allowed.

For example:

```

void transfer(hls::burst_maxi<int> A) {
    #pragma HLS dataflow
    IssueRequests(A); // issue multiple write_request() of A
    Write(A); // multiple writes to A
    GetResponse(A); // write_response() of A
}

```

Potential Pitfalls

The following are some concerns you must be aware of when implementing manual burst techniques:

- **Deadlock:** Improper use of manual burst can lead to deadlocks. Too many `read_requests` before `read()` commands will cause deadlock because the `read_request` loop will push the request into the read requests FIFO, and this FIFO will only be emptied after the read from the global memory is completed. The job of the `read()` command is to read the data from the adapter FIFO and mark the request done, after which the `read_request` will be popped from the FIFO and a new request can be pushed onto it.

```
//reads/writes. will deadlock if N is larger
for (i = 0; i < N; i++)
{
    A.read_request(i * 128, 16);}
for (i = 0; i < 16 * N; i++) { ... = A.read();}
```

```
for (int i = 0; i < N; i++) {
    p.write_request(i * 128, 16);
}
```

```
for (int i = 0; i < N * 16; i++) {
    p.write(i);
}
```

```
for (int i = 0; i < N; i++) {
    p.write_response();
}
```

In the example above, if N is large then the `read_request` and `read` FIFO will be full as it tends to $N/2$. The read request loop would not finish, and the read command loop would not start, which results in deadlock.

 **Note:** This is case also true for `write_request()` and `write()` commands.

- **AXI protocol violation:** There should be an equal number of write requests and write responses. An unequal number of requests and responses would lead to AXI protocol violation

AXI Performance Case Study

Introduction

The objective of the case study is to show a step-by-step optimization to improve the throughput of the read/write loops/functions using HLS metrics. These optimizations will improve the kernel time and throughput of the system by performing efficient data transfers from global memory to the kernel. The `transfer_kernel` example below performs a DDR simple read/write (of variable size and `NUM_ITERATIONS`).

★ **Tip:** The host code, which is not shown, only transfers the data and enqueues the kernel in an in-order queue.

```

1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in,wd* out, const int size, const
int iter ) {
5     ...
6         wd buf[256];
7         int off = (size/16);
8
9         read_loop: for (int i = 0; i < off; i++)
10        {
11            buf[i] = in[i];
12        }
13
14        write_loop: L1: for (int i = 0; i < iter; i++) {
15            L2: for (int j = 0; j < off; j++) {
16                #pragma HLS PIPELINE II=1
17                out[j+off*i] = buf[j];
18            }
19        }
20    ...
21    }
22 }
```

This case study is divided into 4 steps:

1. Baseline kernel run time with port width set to 512-bit width

2. Improve performance by changing latency parameter
3. Improve the auto burst inference of the write loop.
4. No further improvements using multiple ports and number write outstanding

Step 1: Baseline the Kernel with 512-bit Port Width

Baseline the kernel time using the default settings. During this run, the auto burst inferences the following for the read and write loops:

- The Read loop achieves the pipeline burst since the tool can predict the consecutive memory access pattern. So the pipelined requests to read from the DDR of variable size.
- The Write outer loop, L1, gets sequential burst because the compiler iterates over all the combinations and identifies that since the size is unknown at compile-time, it inserts an if condition in the L1 loop before the start of the L2 loop. At the same time, the inner-most loop - L2 achieves pipeline burst. The L2 loop requests a write request of variable size, while L1 waits for all the data of L2 Loop to come back from the DDR to start the next iteration of L1.

After building and running the application, the performance can be evaluated using the Vitis Analyzer tool to view the reports generated by the build process or the run summary. Review the Burst Summary available in the Synthesis Report from Vitis HLS. It confirms the success and failures of the burst for the Read loop and Write loops.

Figure: Synthesis Report - Burst Summary

BURST SUMMARY

HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on

BURSTS AND WIDENING MISSED

HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem	out	VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

In Vitis Analyzer, the Profile Summary and Timeline Trace reports are also useful tools to analyze the performance of the FPGA-accelerated application. In the Profile Summary the Kernels & Compute Unit: Kernel Execution reports the total time required by the transfer_kernel in the baseline build.

Figure: Profile Summary - Kernel Execution

Kernels & Compute Units						
Kernel Execution						
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	
 transfer_kernel	300	1683.150	3.766	5.610	7.947	

Step 2: Improve Performance Latency

Vitis HLS uses the default latency of 64 kernel cycles, which in some cases may be too high. The latency depends on the system characteristics. For this example, the latency is reduced from the default to 21 kernel cycles. The code can be changed to specify the latency using the INTERFACE pragma or directive as shown in the following example:

```

1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in,wd* out, const int size, const
int iter ) {
5         #pragma HLS INTERFACE m_axi port=in0_index offset=slave
latency=21
6         #pragma HLS INTERFACE m_axi port=out offset=slave latency=21
7     ...

```

Build and run the application and use Vitis Analyzer to review the reports generated by the build process or the run summary. Review the Synthesis Report from Vitis HLS, and examine the HW Interface table to see the specified latency has been applied.

Figure: Synthesis Report - HW Interface

M_AXI									
Interface	Data Width (SW->HW)	Address Width	Latency	Offset	Register	Max Widen Bitwidth	Max Read Burst Length	Max Write Burst Length	Num Read Outstanding
m_axi_gmem	512 -> 512	64	21	slave	0	512	16	16	

Review the Burst Summary to examine the success or failures of that process.

Figure: Synthesis Report - Burst Summary 2

BURST SUMMARY

HW Interface	Loop	Message
m_axi_gmem	VITIS_LOOP_11_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on
m_axi_gmem	VITIS_LOOP_17_3	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on

BURSTS AND WIDENING MISSED

HW Interface	Variable	Loop	Problem	Resolution	Location
m_axi_gmem		VITIS_LOOP_16_2	Access call is in the conditional branch	214-232	krnl_tf.cpp:16:22
m_axi_gmem	out		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:17:27
m_axi_gmem	in		Could not widen since the size of type 'i512' is greater than or equal to the max_widen_bitwidth threshold of '64'.		krnl_tf.cpp:11:26

Examine the Kernel Execution in the Profile Summary report, and notice the performance improvement due to setting the latency for the interface.

Figure: Profile Summary - Kernel Execution 2

Kernels & Compute Units

Kernel Execution

Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)
 transfer_kernel	300	1656.180	3.445	5.521	8.002

Step 3: Improve the Automatic Burst Inference of the Write Loop

The compiler is pessimistic in auto burst inference because size and loop trip counts are unknown at compile time. You can modify the code to help the compiler infer pipelined burst, as shown below.

```

1 #include "config.h"
2 #include "assert.h"
3 extern "C" {
4     void transfer_kernel(wd* in, wd* out, const int size, const
int iter ) {
5         #pragma HLS INTERFACE m_axi port=in offset=slave latency=21
6         #pragma HLS INTERFACE m_axi port=out offset=slave latency=21
7
8         int k=0;
9         wd buf[256];
10        int off = (size/16);
11
12        read_loop: for (int i = 0; i <off; i++)
13        {
14            buf[i] = in[i];
15        }

```

```

16
17     write_loop: for (int j = 0; j < off*iter; j++) {
18         #pragma HLS PIPELINE II=1
19         out[k++] = buf[j%off];
20     }
21 }
22 }

```

Build and run the application and use Vitis Analyzer to review the reports generated by the build process or the run summary. The Synthesis Report confirms that the burst hints to the compiler fixed the sequential burst of the write loop. The Burst and Widening Missed messages are related to widening ports to 512 bits. Since this example already has a 512 port width, it can be ignored. If the width isn't 512-bit in your code, you might need to focus on resolving these messages.

Figure: Synthesis Report - Burst Summary 3

BURST SUMMARY			
HW Interface	Loop	Message	
m_axi_gmem	VITIS_LOOP_12_1	Multiple burst reads of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on the burst length.	
m_axi_gmem	VITIS_LOOP_18_2	Multiple burst writes of variable length and bit width 512. These bursts requests might be further partitioned into multiple requests during RTL generation based on the burst length.	

BURSTS AND WIDENING MISSED			
HW Interface	Variable	Problem	Location
m_axi_gmem	out	Could not widen since the size of type '512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:18:27
m_axi_gmem	in	Could not widen since the size of type '512' is greater than or equal to the max_widen_bitwidth threshold of '64'.	krnl_tf.cpp:12:26

Examine the Kernel Execution in the Profile Summary report, and notice the performance improvement due to the latency change from Step 2, and the pipeline burst for the write loop in the current step.

Figure: Profile Summary - Kernel Execution 3

Kernels & Compute Units						
Kernel Execution						
Kernel	Enqueues	Total Time (ms)	Min Time (ms)	Avg Time (ms)	Max Time (ms)	
 transfer_kernel	300	1206.800	1.748	4.023	6.720	

Summary

There are no further improvements that can be made from the Vitis HLS interface metrics. The case study example does not have concurrent read or write, so targeting multiple ports will not help in this case. In this example the tool has achieved pipeline burst for the maximum throughput, so the number of outstanding

reads and writes can also be ignored. No further improvements can be confirmed from the kernel time.

As seen in the case study, implementing efficient load-store functions is dependent on the HLS interface metrics of port width, burst access, latency, multiple ports, and the number of outstanding reads and writes. AMD recommends the following guidelines for improving your system performance:

- Port width: Maximize the port width of the interface, i.e., the bit-width of each AXI port, by using `hls::vector` or `ap_(u)int<512>` as the data type of the port.
- Multiple ports: Analyze the concurrent memory reads/writes and have a dedicated/independent port for concurrent accesses.
- Pipeline burst: The AXI latency parameter does not have an impact on pipelined burst, the user is advised to write code to achieve the pipelined burst which can significantly improve the performance.
- Sequential burst: The AXI latency parameter has a significant impact on sequential burst, decreasing the latency number from the default latency of the tool will improve the performance.
- Num outstanding: In most of the cases of burst length ≥ 16 , the default num outstanding should be sufficient. For a burst of size less than 16, AMD recommends doubling the size of the num outstanding from the default (=16).
- Data Re-ordering: Achieving pipelined burst is always recommended, but at times because of the memory access pattern compiler can achieve only a sequential burst. In order to improve the performance, the developer can also consider different ways of storing the data in memory. For instance, accessing data in DRAM in a column-major fashion can be very inefficient. Rather than implementing a dedicated data-mover in the kernel, it may be better to transpose the data in SW and store in row-major order instead which will greatly simplify HW access patterns.

Managing Area and Hardware Resources

During synthesis, Vitis HLS performs the following basic tasks:

- Elaborates the C, C++ source code into an internal database containing the operators in the C code, such as additions, multiplications, array reads, and writes.
- Maps the operators onto implementations in the hardware.
Implementations are the specific hardware components used to create the design (such as adders, multipliers, pipelined multipliers, and block RAM).

Commands, pragmas and directives provide control over each of these steps, allowing you to control the hardware implementation at a fine level of granularity.

Limiting the Number of Operators

Explicitly limiting the number of operators to reduce area may be required in some cases: the default operation of Vitis HLS is to first maximize performance. Limiting the number of operators in a design is a useful technique to reduce the area of the design: it helps reduce area by forcing the sharing of operations. However, this might cause a decline in performance.

The ALLOCATION directive allows you to limit how many operators are used in a design. For example, if a design called `foo` has 317 multiplications but the FPGA only has 256 multiplier resources (DSP macrocells). The ALLOCATION pragma shown below directs Vitis HLS to create a design with a maximum of 256 multiplication (`mul`) operators:

```
dout_t array_arith (dio_t d[317]) {
    static int acc;
    int i;
    #pragma HLS ALLOCATION instances=fmul limit=256 operation

    for (i=0;i<317;i++) {
    #pragma HLS UNROLL
        acc += acc * d[i];
    }
    rerun acc;
}
```

 **Note:** If you specify an ALLOCATION limit that is greater than needed, Vitis HLS attempts to use the number of resources specified by the limit, or the maximum necessary, which reduces the amount of sharing.

You can use the `type` option to specify if the ALLOCATION directives limits operations, implementations, or functions. The following table lists all the operations

that can be controlled using the ALLOCATION directive.

 **Note:** The operations listed below are supported by the ALLOCATION pragma or directive. The BIND_OP pragma or directive supports a subset of operators as described in the command syntax.

Table: Vitis HLS Operators

Operator	Description
add	Integer Addition
ashr	Arithmetic Shift-Right
dadd	Double-precision floating-point addition
dcmp	Double-precision floating-point comparison
ddiv	Double-precision floating-point division
dmul	Double-precision floating-point multiplication
drecip	Double-precision floating-point reciprocal
drem	Double-precision floating-point remainder
drsqr	Double-precision floating-point reciprocal square root
dsub	Double-precision floating-point subtraction
dsqr	Double-precision floating-point square root
fadd	Single-precision floating-point addition
fcmp	Single-precision floating-point comparison
fdiv	Single-precision floating-point division
fmul	Single-precision floating-point multiplication
frecip	Single-precision floating-point reciprocal
frem	Single-precision floating point remainder
frsqr	Single-precision floating-point reciprocal square root
fsub	Single-precision floating-point subtraction

Operator	Description
fsqrt	Single-precision floating-point square root
icmp	Integer Compare
lshr	Logical Shift-Right
mul	Multiplication
sdiv	Signed Divider
shl	Shift-Left
srem	Signed Remainder
sub	Subtraction
udiv	Unsigned Division
urem	Unsigned Remainder

Controlling Hardware Implementation

When synthesis is performed, Vitis HLS uses the timing constraints specified by the clock, the delays specified by the target device together with any directives specified by you, to determine which hardware implementations to use for various operators in the code. For example, to implement a multiplier operation, Vitis HLS could use the combinational multiplier or use a pipeline multiplier.

The implementations which are mapped to operators during synthesis can be limited by specifying the `ALLOCATION` pragma or directive, in the same manner as the operators. Instead of limiting the total number of multiplication operations, you can choose to limit the number of combinational multipliers, forcing any remaining multiplications to be performed using pipelined multipliers (or vice versa).

The `BIND_OP` or `BIND_STORAGE` pragmas or directives are used to explicitly specify which implementations to use for specific operations or storage types. The following command informs Vitis HLS to use a two-stage pipelined multiplier using fabric logic for variable `c`. It is left to Vitis HLS which implementation to use for variable `d`.

```
int foo (int a, int b) {
    int c, d;
```

```
#pragma HLS BIND_OP variable=c op=mul impl=fabric latency=2
c = a*b;
d = a*c;

return d;
}
```

In the following example, the BIND_OP pragma specifies that the add operation for variable temp is implemented using the dsp implementation. This ensures that the operation is implemented using a DSP module primitive in the final design. By default, add operations are implemented using LUTs.

```
void apint_arith(dinA_t inA, dinB_t inB,
                dout1_t *out1
                ) {

    dout2_t temp;
    #pragma HLS BIND_OP variable=temp op=add impl=dsp

    temp = inB + inA;
    *out1 = temp;

}
```

Refer to the BIND_OP or BIND_STORAGE pragmas or directives to obtain details on the implementations available for assignment to operations or storage types. In the following example, the BIND_OP pragma specifies the multiplication for out1 is implemented with a 3-stage pipelined multiplier.

```
void foo(...) {
    #pragma HLS BIND_OP variable=out1 op=mul latency=3

    // Basic arithmetic operations
    *out1 = inA * inB;
    *out2 = inB + inA;
    *out3 = inC / inA;
    *out4 = inD % inA;

}
```

If the assignment specifies multiple identical operators, the code must be modified to ensure there is a single variable for each operator to be controlled. For example, in the following code, if only the first multiplication (`inA * inB`) is to be implemented with a pipelined multiplier:

```
*out1 = inA * inB * inC;
```

The code should be changed to the following with the pragma specified on the `Result_tmp` variable:

```
#pragma HLS BIND_OP variable=Result_tmp op=mul latency=3
Result_tmp = inA * inB;
*out1 = Result_tmp * inC;
```

DSP Multi-Operation Matching

The HLS compiler attempts to identify Add-Multiply, Multiply-Add, and AMA operations (as well as Subtract-Multiply, Multiply-Subtract operations) to match their assignment to DSPs on the physical device. The HLS compiler uses the following process to match operations to DSPs:

- Inline functions as needed
- Extract common sub-expressions
- Match fanout-free `add->mul`, `mul->add`, and `add->mul->add` (where `add` can also be `sub`) code fragments meeting bitwidth limitations of DSPs
- Schedule
- Select implementation in DSP or LUT guided by the `BIND_OP` pragma

The ability of the HLS compiler to match operations or expressions to DSP implementation depends on the following:

1. Datatypes and bitwidths
2. Coding style and expressions
3. The use of `BIND_OP` and other pragmas

Datatypes and Bitwidths

DSP matching is limited by design restrictions of DSP48 and DSP58 on the target device. Refer to *UltraScale Architecture DSP Slice* ([UG579](#)) for specific limitations

on DSP48, and *Versal Adaptive SoC DSP Engine Architecture Manual* (AM004) for limits on DSP58.

As an example, in Versal devices the following are the port widths for DSP58:

- B 24 bits
- A 34 bits
- D 27 bits
- C 58 bits
- P 58 bits

Coding Style

The style of writing the code matters:

- Bitwidths must be small enough to fit the DSP, including intermediate results
- No fanout is allowed, even as a result of common sub-expression elimination. For example the following code will not be matched for DSP assignment due to the extraction of the sub-expression $a*b$:

```
f1 = a*b+c;
f2= a*b+d;
```

★ **Tip:** To avoid this problem use non-inlined functions containing exactly the expressions to be matched.

- Non-inlined functions with specified latency, for example MULADD with latency 2, cannot be matched to an accumulator with $II=1$. The reason is that both multiplier input and the adder input must be ready 2 cycles before the accumulation output is produced. In that case, the MULADD or MAC function (if any) must be inlined.

BIND_OP and Other Pragmas

For DSP matching, some pragmas or directives can prevent the matching of code for implementation in DSPs, while other pragmas or directives can help. In the following code example, the BIND_OP pragma actually prevents the assignment of the MULADD expression to DSP. By assigning the mul operation it prevents the HLS compiler from recognizing and assigning the MULADD expression. Removing the pragma in this case will let the compiler make the assignment.

```

static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC, 3, AP_TRN, AP_WRAP> DATA_T;
...
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T acc ) {
    acc = din0 * coef + acc;
#pragma HLS BIND_OP variable=acc op=mul impl=dsp latency=2
    return acc;
}
void process ( DATA_T din, DATA_T* dout0) {
#pragma HLS pipeline II=1
...
    LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
        acc = MAC (sr[tdl_index], coeff[i], acc);
        tdl_index = tdl_index-1;
    }
}

```

The PIPELINE pragma causes the loop to be unrolled with expression balancing that also prevents DSP matching. In this case, adding EXPRESSION_BALANCE OFF to disable expression balancing will help the HLS compiler recognize the multi-operation expressions for DSP matching. The reworked code example below will have improved DSP matching.

```

static const int NFRAC = 14;
typedef ap_fixed<3+NFRAC, 3, AP_TRN, AP_WRAP> DATA_T;
...
ACC_T MAC( DATA_T din0, COEF_T coef, ACC_T acc ) {
    acc = din0 * coef + acc;
    return acc;
}
void process ( DATA_T din, DATA_T* dout0) {
#pragma HLS pipeline II=1
#pragma HLS expression_balance off
...
    LOOP_MAC: for (int i=0; i<l_COEFF; i++) {
        acc = MAC (sr[tdl_index], coeff[i], acc);
        tdl_index = tdl_index-1;
    }
}

```

Controlling Operator Pipelining

The HLS compiler automatically determines the level of pipelining to use for internal operations. You can use the `BIND_OP` or `BIND_STORAGE` pragmas with the `-latency` option to explicitly specify the number of pipeline stages and override the number determined by the tool.

RTL synthesis might use the additional pipeline registers to help improve timing issues that might result after place and route. Registers added to the output of the operation typically help improve timing in the output datapath. Registers added to the input of the operation typically help improve timing in both the input datapath and the control logic from the FSM.

You can use the Operator Configuration commands to pipeline all instances of a specific operation used in the design that have the same pipeline depth.

Unrolling Loops in C++ Classes

!! Important: When loops are used in C++ classes, care should be taken to ensure that the loop induction variable is not a data member of the class as this prevents the loop from being unrolled.

In this example, loop induction variable `k` is a member of class `loop_class`.

```
template <typename T0, typename T1, typename T2, typename T3, int
N>
class loop_class {
private:
    pe_mac<T0, T1, T2> mac;
public:
    T0 areg;
    T0 breg;
    T2 mreg;
    T1 preg;
    T0 shift[N];
    int k;                // Class Member
    T0 shift_output;
    void exec(T1 *pcout, T0 *dataOut, T1 pcin, T3 coeff, T0 data, int
col)
    {
Function_label0:;
#pragma HLS inline off
        SRL:for (k = N-1; k >= 0; --k) {
```

```
#pragma HLS unroll // Loop will fail UNROLL
if (k > 0)
  shift[k] = shift[k-1];
else
  shift[k] = data;
}

    *dataOut = shift_output;
    shift_output = shift[N-1];
}

*pcout = mac.exec1(shift[4*col], coeff, pcin);
};
```

For Vitis HLS to be able to unroll the loop as specified by the UNROLL pragma directive, the code should be rewritten to remove k as a class member and make it local to the exec function.

Limitations of Control-Driven Task-Level Parallelism

★ **Tip:** Control-driven TLP requires the DATAFLOW pragma or directive to be specified in the appropriate location of the code.

The control-driven TLP model optimizes the flow of data between tasks (functions and loops), and ideally pipelined functions and loops for maximum performance. It does not require these tasks to be chained, one after the other, however there are some limitations in how the data is transferred. The following behaviors can prevent or limit the overlapping that the tool can perform in the dataflow model:

- Reading from function inputs or writing to function outputs in the middle of the dataflow region
- Single-producer-consumer violations
- Conditional execution of tasks
- Loops with multiple exit conditions

!! Important: If any of these coding styles are present, the HLS compiler issues a message describing the situation.

Reading from Inputs/Writing to Outputs

Reading of inputs of the function should be done at the start of the dataflow region, and writing to outputs should be done at the end of the dataflow region. Reading/writing to the ports of the function can cause the processes to be executed in sequence rather than in an overlapped fashion, adversely impacting performance.

Single-producer-consumer Violations

For the tool to use the dataflow model, all elements passed between tasks must follow a single-producer-consumer model. Each variable must be driven from a single task and only be consumed by a single task. In the following code example, temp1 fans out and is consumed by both Loop2 and Loop3. This violates the single-producer-consumer model.

```
void foo(int data_in[N], int scale, int data_out1[N], int
data_out2[N]) {
    int temp1[N];

    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
    }
    Loop2: for(int j = 0; j < N; j++) {
        data_out1[j] = temp1[j] * 123;
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out2[k] = temp1[k] * 456;
    }
}
```

A modified version of this code uses function Split to create a single-producer-consumer design. The following code block example shows how the data flows with the function Split. The data now flows between all four tasks, and the tool can use the dataflow model.

```
void Split (in[N], out1[N], out2[N]) {
// Duplicated data
L1:for(int i=1;i<N;i++) {
```

```

    out1[i] = in[i];
    out2[i] = in[i];
}
}
void foo(int data_in[N], int scale, int data_out1[N], int
data_out2[N]) {

    int temp1[N], temp2[N], temp3[N];
    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
    }
    Split(temp1, temp2, temp3);
    Loop2: for(int j = 0; j < N; j++) {
        data_out1[j] = temp2[j] * 123;
    }
    Loop3: for(int k = 0; k < N; k++) {
        data_out2[k] = temp3[k] * 456;
    }
}

```

Bypassing Tasks and Channel Sizing

In addition, data should generally flow from one task to another. If you bypass tasks, this can reduce the performance of the dataflow model. In the following example, Loop1 generates the values for temp1 and temp2. However, the next task, Loop2, only uses the value of temp1. The value of temp2 is not consumed until *after* Loop2. Therefore, temp2 bypasses the next task in the sequence, which can limit the performance of the dataflow model.

```

void foo(int data_in[N], int scale, int data_out1[N], int
data_out2[N]) {
    int temp1[N], temp2[N], temp3[N];
    Loop1: for(int i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
        temp2[i] = data_in[i] >> scale;
    }
    Loop2: for(int j = 0; j < N; j++) {
        temp3[j] = temp1[j] + 123;
    }
    Loop3: for(int k = 0; k < N; k++) {

```

```

    data_out[k] = temp2[k] + temp3[k];
  }
}

```

In this case, you should increase the depth of the PIPO buffer used to store temp2 to be 3, instead of the default depth of 2. This lets the buffer store the value intended for Loop3, while Loop2 is being executed. Similarly, a PIPO that bypasses two processes should have a depth of 4. Set the depth of the buffer with the STREAM pragma or directive:

```
#pragma HLS STREAM type=pipo variable=temp2 depth=3
```

!! Important: Channel sizing can also similarly affect performance. Having mismatched FIFO/PIPO depths can inadvertently cause synchronization points inside the dataflow region because of back pressure from the FIFO/PIPO.

Feedback between Tasks

Feedback occurs when the output from a task is consumed by a previous task in the dataflow region. Non-stream feedback channels (For example, scalars or arrays) are not supported in a dataflow region. When the HLS compiler detects feedback, it issues a warning, depending on the situation, and might not use the dataflow model.

However, dataflow can support feedback when used with `hls::streams`. The following example demonstrates this exception.

```

#include "ap_axi_sdata.h"
#include "hls_stream.h"

void firstProc(hls::stream<int> &forwardOUT, hls::stream<int>
&backwardIN) {
    static bool first = true;
    int fromSecond;

    //Initialize stream
    if (first)
        fromSecond = 10; // Initial stream value
    else
        //Read from stream
        fromSecond = backwardIN.read(); //Feedback value
}

```

```

first = false;

//Write to stream
forwardOUT.write(fromSecond*2);
}

void secondProc(hls::stream<int> &forwardIN, hls::stream<int>
&backwardOUT) {
    backwardOUT.write(forwardIN.read() + 1);
}

void top(...) {
#pragma HLS dataflow
    hls::stream<int> forward, backward;
    firstProc(forward, backward);
    secondProc(forward, backward);
}

```

In this simple design, when `firstProc` is executed, it uses 10 as an initial value for input. Because `hls::streams` do not support an initial value, this technique can be used to provide one without violating the single-producer-consumer rule. In subsequent iterations `firstProc` reads from the `hls::stream` through the `backwardIN` interface.

`firstProc` processes the value and sends it to `secondProc`, via a stream that goes *forward* in terms of the original C++ function execution order. `secondProc` reads the value on `forwardIN`, adds 1 to it, and sends it back to `firstProc` via the feedback stream that goes *backwards* in the execution order.

From the second execution, `firstProc` uses the value read from the stream to do its computation, and the two processes can keep going forever, with both forward and feedback communication, using an initial value for the first execution.

Conditional Execution of Tasks

The dataflow model does not optimize tasks that are conditionally executed. The following example highlights this limitation. In this example, the conditional execution of `Loop1` and `Loop2` prevents the tool from optimizing the data flow between these loops, because the data does not flow from one loop into the next.

```

void foo(int data_in1[N], int data_out[N], int sel) {

```

```

int temp1[N], temp2[N];

if (sel) {
Loop1: for(int i = 0; i < N; i++) {
temp1[i] = data_in[i] * 123;
temp2[i] = data_in[i];
}
} else {
Loop2: for(int j = 0; j < N; j++) {
temp1[j] = data_in[j] * 321;
temp2[j] = data_in[j];
}
}
Loop3: for(int k = 0; k < N; k++) {
data_out[k] = temp1[k] * temp2[k];
}
}

```

To ensure each loop is executed in all cases, you must transform the code as shown in the following example. In this example, the conditional statement is moved into the first loop. Both loops are always executed, and data always flows from one loop to the next.

```

void foo(int data_in[N], int data_out[N], int sel) {

int temp1[N], temp2[N];

Loop1: for(int i = 0; i < N; i++) {
if (sel) {
temp1[i] = data_in[i] * 123;
} else {
temp1[i] = data_in[i] * 321;
}
}
Loop2: for(int j = 0; j < N; j++) {
temp2[j] = data_in[j];
}
Loop3: for(int k = 0; k < N; k++) {
data_out[k] = temp1[k] * temp2[k];
}
}

```

Loops with Multiple Exit Conditions

Loops with multiple exit points cannot be used in a dataflow region. In the following example, Loop2 has three exit conditions:

- An exit defined by the value of N; the loop will exit when $k \geq N$.
- An exit defined by the break statement.
- An exit defined by the continue statement.

```
#include "ap_int.h"
#define N 16

typedef ap_int<8> din_t;
typedef ap_int<15> dout_t;
typedef ap_uint<8> dsc_t;
typedef ap_uint<1> dsel_t;

void multi_exit(din_t data_in[N], dsc_t scale, dsel_t select,
dout_t data_out[N]) {
    dout_t temp1[N], temp2[N];
    int i,k;

    Loop1: for(i = 0; i < N; i++) {
        temp1[i] = data_in[i] * scale;
        temp2[i] = data_in[i] >> scale;
    }

    Loop2: for(k = 0; k < N; k++) {
        switch(select) {
            case 0: data_out[k] = temp1[k] + temp2[k];
            case 1: continue;
            default: break;
        }
    }
}
```

Because a loop's exit condition is always defined by the loop bounds, the use of break or continue statements will prohibit the loop being used in a DATAFLOW region.

Finally, the dataflow model has no hierarchical implementation. If a sub-function or loop contains additional tasks that might benefit from dataflow, you must apply the dataflow model to the loop, the sub-function, or inline the sub-function.

You can also use `std::complex` inside the dataflow region. However, they should be used with an `__attribute__((no_ctor))` as shown in the following example:

```

void proc_1(std::complex<float> (&buffer)[50], const
std::complex<float> *in);
void proc_2(hls::Stream<std::complex<float>> &fifo, const
std::complex<float> (&buffer)[50], std::complex<float> &acc);
void proc_3(std::complex<float> *out,
hls::Stream<std::complex<float>> &fifo, const std::complex<float>
acc);

void top(std::complex<float> *out, const std::complex<float> *in) {
#pragma HLS DATAFLOW

    std::complex<float> acc __attribute__((no_ctor)); // Here
    std::complex<float> buffer[50] __attribute__((no_ctor)); // Here
    hls::Stream<std::complex<float>, 5> fifo; // Not here

    proc_1(buffer, in);
    proc_2(fifo, buffer, acc);
    proc_3(out, fifo, acc);
}

```

Limitations of Pipelining with Static Variables

Static variables are used to keep data between loop iterations, often resulting in registers in the final implementation. If this is encountered in pipelined functions, Vitis HLS might not be able to optimize the design sufficiently, which would result in initiation intervals longer than required.

The following is a typical example of this situation:

```

function_foo()
{
    static bool change = 0
    if (condition_xyz){
        change = x; // store
    }
    y = change; // load
}

```

If Vitis HLS cannot optimize this code, the stored operation requires a cycle and the load operation requires an additional cycle. If this function is part of a pipeline, the

pipeline has to be implemented with a minimum initiation interval of 2 as the static change variable creates a loop-carried dependency.

One way the user can avoid this is to rewrite the code, as shown in the following example. It ensures that only a read or a write operation is present in each iteration of the loop, which enables the design to be scheduled with $II=1$.

```
function_readstream()  
{  
    static bool change = 0  
    bool change_temp = 0;  
    if (condition_xyz)  
    {  
        change = x; // store  
        change_temp = x;  
    }  
    else  
    {  
        change_temp = change; // load  
    }  
    y = change_temp;  
}
```